CONTENTS

TOPICS **PAGE**
TEMPLATE....1
TEMPLATE JAVA....1
BIT OPERATIONS....1
DATA STRUCTURES
SEGMENT TREE....2
PERSISTENT SEGMENT TREE....2
BINARY INDEXED (FENWICK) TREE....2
AVL TREE....2
LINK CUT TREE....3
TRIES....4
MO ALGORITHM....4
MISOF TREE....4
HEAVY LIGHT DECOMPOSITION....5
DYNAMIC PROGRAMMING
LIS (LONGEST INCREASING SUBSEQUENCE)....5
CYCLIC LCS (LONGEST COMMON SUBSEQUENCE)....6
LCIS (LONGEST COMMON INCREASING SUBSEQUENCE)....6
DP Buka String & Lipat String....6
MATRIX CHAIN MULTIPLICATION....7
MIN CUT POLYGON TRIAGULATION....7
KADANE....7
DP SOS....7
DP CONVEX HULL....7
DP DIVIDE AND CONQUER....7
DP KNUTH YAO....7
GRAPHS
TOPOLOGICAL SORT....8
K-th SHORTEST PATH....8
MANHATTAN SPANNING TREE....8
DIRECTED MST....8
BELLMAN FORD (Negative Cycle Detection)....9
DINIC ALGORITHM....9
LOWEST COMMON ANCESTOR (SEG TREE)....9
MIN COST MAX FLOW....10
MIN COST MAX FLOW (Multiple Edge)....10
MAXFLOW (AdJ List)....10
MIN CUT (Stoer Wagner)....11
EULERIAN TOUR CHECKING....11
MAX BIPARTITE MATCHING....11
MINIMUM VERTEX COVER....11
ARTICULATION POINT & BRIDGE....12
TARJAN ALGORITHM....12
HOPCROFT KARP....12
EULER PATH + EULER TOUR....12
EDMONDS BLOSSOM....13
AHO CORASICK....13
MAXIMUM CLIQUE....13
DSU ON TREE....14
RECOVER MIN-VERTEX COVER from MAXIMUM MATCHING....14
STRING PROCESSING
KMP ALGORITHM....14
SUFFIX ARRAY....14
ENHANCED SUFFIX ARRAY....14
EDIT DISTANCE....15
K-th PERMUTATION....15
ETC
Soal TIMO 2010....15
MEET IN THE MIDDLE....16
HUFFMAN CODES....16
HUNGARIAN ALGORITHM....16
STABLE MARRIAGE....16
CSOD....16
RDN (GOTO 01/01/0001)....16
MATHEMATICS
BIG INTEGER....16
FAST FOURIER TRANSFORM + NTT....18
GAUSS ELIMINATION....18
NUMBER THEORY....18
MATHEMATICS SEQUENCE FORMULA....19
PRIME NUMBER LIST....19
CATALAN & SUPER CATALAN....19
DIGIT COUNTING FACTORIAL....19
INT TO ROMAN....19
nCk....19
LINEAR DIOPHANTINE....19
CRT dan kawan2....20
EULER TOTIENT (PHI) FUNCTION....20
PRIME TEST....20
POLLARD RHO....21
MILLER RABIN....21
GENERALIZED LUCAS THEOREM....21
EXTENDED EUCLID....21
SUM OF DIVISORS....21
RANGE SIEVE....21
MÖBIUS FUNCTION....21
GEOMETRIC TRANSFORMATION....21
TRIGONOMETRIC EQUATIONS....21
GEOMETRY
POLYGON CENTROID AND AREA....22
LINE SEGMENT INTERSECTION POINT....22
LINE SEGMENT INTERSECTION....22
SQUARE COLLISION....22
CLOSEST PAIR....22
POLYGON CLASS....23
PT TO LINE & LINE TO SEGMENT DIST....23
POLYGON DIAMETER....24
UNION RECTANGLE INTERSECTION....24
GREAT CIRCLE DISTANCE....24
END OF TEAM NOTEBOOK....24

END OF CONTENTS

TEMPLATE

```
#pragma GCC optimize("O3")
const int OO=(int)1e9;
const ll INF=(ll)2e18;
#define FOR(i,n) for(int i=0;i<n;i++)
#define REPP(i,l,r,c) for(int i=l;i<=r;i+=c)
#define REP(i,l,r) REPP(i,l,r,1)
#define FORD(i,n) for(int i=n-1;i>=0;i--)
#define REVV(i,l,r,c) for(int i=l;i>=r;i-=c)
#define REV(i,l,r) REVV(i,l,r,1)

int irand(int lo,int hi){// random
  return((double)rand()/(RAND_MAX+1.0))*(hi-lo+1)+lo;
}
string toString(ll x){ // ll to string
  stringstream ss;  ss << x;  return ss.str();
}
ll toNumber(string S){ // string to ll
  ll ret;  sscanf(S.c_str(),"%lld",&ret); return ret;
}
void ioSync(){ //iostream sync
  ios_base::sync_with_stdio(0);
  cin.tie(0);  cout.tie(0);
}
```

TEMPLATE JAVA

```
import java.math.BigInteger;
import java.util.Scanner;
import java.util.ArrayList;

class Main{
  static ArrayList A=new ArrayList<BigInteger>();
  static void init(){
    BigInteger x = new BigInteger("1656");
    BigInteger y = BigInteger.valueOf(123);
    x = x.add(new BigInteger("4498"));
    // substract  multiply  divide  remainder
    // compare -> (A.compareTo(B)<0) (A.equals(B))
    int x = A.intValue();
    long y = A.longValue();
    String s = A.toString();
  }
  public static void Main(String[] args){
    init();  int x;
    Scanner in = new Scanner(System.in);
    while(in.hasNext())
      System.out.println("Print "+in.NextInt());
} }
```

BIT OPERATIONS

```
int A[30];//Bilangan 2^N
void prework(){//Precompute 2^N
  A[0] = 1; REP(i,1,30)A[i] = A[i-1]*2;
}
int getbit(int num, int pos){ return (num&A[pos])!=0;}
void setbit(int& num, int pos, int biner){
  //Set bit ke POS jadi BINER di NUM
  int temp = getBit(num,pos);
```

```
  if(temp!=biner)num+=((temp<biner)?A[pos]:-A[pos]);
}
int removeLSOne(int num){ return num&(num-1); }
__builtin_popcount(x)// count ON bit
__builtin_clz(x)// number of leading 0
__builtin_ctz(x)// get number of trailing 0
(x&(1<<y))// sisain bit ke-y+1  T=(S&(-S))bit pertama
x&=~(1<<y)// matiin bit ke-y+1  yg nyala dari kanan
```

SEGMENT TREE

```
//array size: 2N        data inside N..2N-1

void build(){// build the tree
  REV(i,n-1,0)t[i] = t[i<<1]+t[i<<1|1];
}
void updPURQ(int pos, int val){
  for(t[pos+=n] = value; pos>1; pos>>=1)
    t[pos>>1] = t[pos]+t[pos^1];
}
int queryPURQ(int l, int r){
  int res = 0;
  for(l+=n,r+=n; l<r; l>>=1,r>>=1){
    if(l&1)res += t[l++];  if(r&1)res += t[--r];
  }
  return res;
}
void updRUPQ(int l, int r, int val){
  for(l+=n,r+=n; l<r; l>>=1,r>>=1){
    if(l&1)t[l++] += val;
    if(r&1)t[--r] += val;
} }
int queryRUPQ(int pos){
  int res = 0;
  for(pos+=n; pos>0; pos>>=1)res += t[pos];
  return res;
}
void pushRUPQ(){//inspect all elements: O(n)
  REP(i,1,n-1){
    t[i<<1] += t[i];  t[i<<1|1] += t[i];  t[i] = 0;
} }
// h = height of the tree (log)
void propoRURQ(int pos, int val){ //propagate lazy
  t[pos]+=val;
  if(pos<n)lazy[pos]+=value;
}
void updParRURQ(int pos){//update all parents
  while(pos>1){
    pos >>= 1;
    t[pos] = max(t[pos<<1],t[pos<<1|1]) + lazy[pos];
} }
void pushRURQ(int p){//propagates from ancestor
  REV(i,h,1){
    int pos = p>>i;
    if(lazy[pos]){
      propoRURQ(pos<<1,lazy[pos]);
      propoRURQ(pos<<1|1,lazy[pos]);
      lazy[pos] = 0;
} } }
void updRURQ(int l, int r, int val){
  l+=n;  r+=n;  int l0 = l, r0 = r;
  for(; l<r; l>>=1,r>>=1){
    if(l&1)propoRURQ(l++,val);
    if(r&1)propoRURQ(--r,val);
  }
  updParRURQ(l0);  updParRURQ(r0-1);
}
int queryRURQ(int l, int r){
  l+=n;  r+=n;  pushRURQ(l);  pushRURQ(r-1);
  int res = -2e9;
  for(; l < r; l >>=1, r >>=1){
    if(l&1)res = max(res,t[l++]);
    if(r&1)res = max(res,t[--r]);
  }
  return res;
}
```

PERSISTENT SEGMENT TREE

```
struct Tree{
  int dat,L,R;
  Tree(){ dat = 0; L = R = -1; }
} seg[2400005];
int num =0;
int dat[100005], temp[100005], headList[100005];
map<int,int> hashh;  int rehashh[100005];

void build(int idx, int l, int r){
  if(l==r)return;
  int mid=(l+r)>>1;
  build(seg[idx].L=++num,l,mid);
  build(seg[idx].R=++num,mid+1,r);
}
void upd(int idx, int l, int r, int key, int prev){
  if(l==r){ seg[idx].dat=seg[prev].dat+1;  return; }
  int mid=(l+r)>>1;  seg[idx]=seg[prev];
  if(key<=mid)
    upd(seg[idx].L=++num,l,mid,key,seg[prev].L);
  else upd(seg[idx].R=++num,mid+1,r,key,seg[prev].R);
 seg[idx].dat=seg[seg[idx].L].dat+seg[seg[idx].R].dat;
}
int que(int idx, int prev, int l, int r, int kth){
  if(l==r)return l;
  int LF=seg[seg[idx].L].dat-seg[seg[prev].L].dat;
  int mid=(l+r)>>1;
  if(kth<=LF)
    return que(seg[idx].L,seg[prev].L,l,mid,kth);
  return que(seg[idx].R,seg[prev].R,mid+1,r,kth-LF);
}
int main(){
  FOR(i,100003)dat[i]=0,headList[i]=rehashh[i]=-1;
  hashh.clear();
  //input
  int n,q, idx=0;
  scanf("%d%d",&n,&q);
  FOR(i,n)scanf("%d",dat+i), temp[i]=dat[i];
  sort(temp,temp+n);  temp[n]=temp[n-1]+1;
  FOR(i,n) if(temp[i]!=temp[i+1])
    hashh[temp[i]]=++idx,rehashh[idx]=temp[i];
  //build
  build(headList[0]=++num,1,idx);
  FOR(i,n){
    headList[i+1]=++num;
    upd(num,1,idx,hashh[dat[i]],headList[i]);
  }
  //query
  while(q--){
    int l,r,kth;
    scanf("%d%d%d",&l,&r,&kth);
    r=headList[r];  l=headList[l-1];
    printf("%d\n",rehashh[que(r,l,1,idx,kth)]);
} }
```

BINARY INDEXED (FENWICK) TREE

```
int LSOne(int n){ return(n&-n); }
void upd(int* dat, int pos, int size, int value){
  REPP(i,pos,n,LSOne(i))dat[i]+=value; // i|=i+1
}
int que(int* dat, int pos){
  int res =0;
  REVV(i,pos,1,LSOne(i))res+=dat[i]; // b=(b&(b+1))-1
  return res;
}
```

Range Update

```
upd(BITree1, l, val);      upd(BITree2, l, val*(l-1));
upd(BITree1, r+1, -val);   upd(BITree2, r+1, -val*r);
```

Range [1..k] Query Sum

```
que(BITTree1, k)* k - que(BITTree2, k);
```

AVL TREE

```
struct Tree{
  int key, ubound, h, sz;  Tree *lf,*rg;
  Tree(int key){
    this->key=key;  lf = rg = NULL;
    this->ubound=0;  h = 0;  sz = 1;
  }
}*root;
int height(Tree *p){ return p==NULL ? -1 : p->h; }
int size(Tree *p){ return p==NULL ? 0 : p->sz; }
int upper_bound(Tree *p,int x){
  if(p == NULL)return 0;
  if(p->key<=x)return p->ubound = size(p->lf)+1+    \
    upper_bound(p->rg,x);
  return upper_bound(p->lf,x);
```

```
}
void rotateL(Tree *&p){
  Tree *c=p->lf;  p->lf=c->rg;  c->rg=p;
  p->h=max(height(p->lf),height(p->rg))+1;
  c->h=max(height(c->lf),p->h)+1;
  p->sz=size(p->lf)+ size(p->rg)+1;
  c->sz=size(c->lf)+ size(c->rg)+1;
  p=c;
}
void rotateR(Tree *&p){
  Tree *c=p->rg;  p->rg=c->lf;  c->lf=p;
  p->h=max(height(p->lf),height(p->rg))+1;
  c->h=max(height(c->rg),p->h)+1;
  p->sz=size(p->lf)+ size(p->rg)+1;
  c->sz=size(c->lf)+ size(c->rg)+1;
  p=c;
}
void insert(int x, Tree *&p){
  if(p==NULL)p=new Tree(x);
  else if(x<=p->key){
    insert(x,p->lf);
    if(height(p->lf)-height(p->rg)>=2){
      if(x>=p->lf->key)rotateR(p->lf);
      rotateL(p);
  } }
  else if(x>p->key){
    insert(x,p->rg);
    if(height(p->rg)-height(p->lf)>=2){
      if(x<=p->rg->key)rotateL(p->rg);
      rotateR(p);
  } }
  p->h=max(height(p->lf),height(p->rg))+1;
  p->sz=size(p->lf)+ size(p->rg)+1;
  p->ubound=upper_bound(p,p->key);
}
int kth(Tree *p,int x){
  if(size(p->lf)+1==x)return p->key;
  if(x < size(p->lf)+1)return kth(p->lf,x);
  return kth(p->rg,x-size(p->lf)-1);
}
void printAll(Tree *p){
  if(p==NULL)return;
  printAll(p->lf);
  printf(" %d ",p->key);
  printAll(p->rg);
}
Tree* findMin(Tree *p){
  while(p->lf!=0)p = p->lf;  return p;
}
void removeAll(Tree *p){
  if(p==NULL)return;
  removeAll(p->lf);  removeAll(p->rg);  delete p;
}
void remove(int x, Tree *&p){
  if(p==0)return;
  if(x<p->key)remove(x,p->lf);
  else if(x>p->key)remove(x,p->rg);
  else if(p->lf!=0&& p->rg!=0){
    p->key = findMin(p->rg)->key;
    remove(p->key,p->rg);
  }
  else{
    Tree *temp = p;  p = (p->lf!=0)? p->lf:p->rg;
    delete temp;
  }
  if(p!=0){
    p->sz = size(p->lf)+ size(p->rg)+1;
    p->h = max(height(p->lf),height(p->rg))+1;
} }
```

LINK CUT TREE

```
// LinkCut tree(n);// new link-cut tree with n nodes
// tree.link(x, y);// link x and y
// tree.cut(x);  // cut x
// tree.root(x); // root of tree containing x
// tree.depth(x);  // depth of x in its tree
// tree.lca(x, y); // LCA of x and y
struct Node{
  int sz, label; // size, label
  Node *p,*pp;   // parent, path-parent
  Node *l,*r;    // left, right pointers
  Node(){ p = pp = l = r =0; }
};
void update(Node *x){
  x->sz = 1;
  if(x->l)x->sz += x->l->sz;
  if(x->r)x->sz += x->r->sz;
}
void rotr(Node *x){
  Node *y,*z;  y = x->p, z = y->p;
  if((y->l = x->r))y->l->p = y;
  x->r = y, y->p = x;
  if((x->p = z)){
    if(y == z->l) z->l = x;
    else z->r = x;
  }
  x->pp = y->pp;  y->pp = 0;  update(y);
}
void rotl(Node *x){
  Node *y,*z;y = x->p, z = y->p;
  if((y->r = x->l))y->r->p = y;
  x->l = y, y->p = x;
  if((x->p = z)){
    if(y == z->l) z->l = x;
    else z->r = x;
  }
  x->pp = y->pp;  y->pp =0;  update(y);
}
void splay(Node *x){
  Node *y,*z;
  while(x->p){
    y = x->p;
    if(y->p ==0){
      if(x == y->l) rotr(x);
      else rotl(x);
    }
    else{
      z = y->p;
      if(y == z->l){
        if(x == y->l) rotr(y), rotr(x);
        else rotl(x), rotr(x);
      }
      else{
        if(x == y->r) rotl(y), rotl(x);
        else rotr(x), rotl(x);
  } } }
  update(x);
}
Node *access(Node *x){
  splay(x);
  if(x->r){
    x->r->pp = x;  x->r->p =0;  x->r =0;  update(x);
  }
  Node *last = x;
  while(x->pp){
    Node *y = x->pp;  last = y;  splay(y);
    if(y->r) y->r->pp = y, y->r->p =0;
    y->r = x;  x->p = y;  x->pp =0;
    update(y);  splay(x);
  }
  return last;
}
Node *root(Node *x){
  access(x);  while(x->l) x = x->l;  splay(x);
  return x;
}
void cut(Node *x){
  access(x);  x->l->p = x->l = 0;  update(x);
}
void link(Node *x, Node *y){
  access(x);  access(y);  x->l = y;  y->p = x;
  update(x);
}
Node *lca(Node *x, Node *y){
  access(x);  return access(y);
}
int depth(Node *x){
  access(x);  return x->sz -1;
}
class LinkCut{
  Node *x;
```

```
  public:
  LinkCut(int n){
    x = new Node[n];
    FOR(i,n){ x[i].label = i;  update(&x[i]); }
  }
  virtual ~LinkCut(){ delete[] x; }
  void link(int u,int v){ ::link(&x[u],&x[v]); }
  void cut(int u){ ::cut(&x[u]); }
  int root(int u){ return ::root(&x[u])->label; }
  int depth(int u){ return ::depth(&x[u]); }
  int lca(int u, int v){
    return ::lca(&x[u],&x[v])->label;
} };
```

TRIES

```
typedef struct{
  char kar;  bool isWord;  int parent;
  map<char,int> child;
} node;
char s[100];  node tree[2000001];  int idx,len,nn,ma;

void insert(int nIdx, int cIdx){ // nodeIdx, charIdx
  if(tree[nIdx].child.count(s[cIdx])==0){
    tree[++idx].kar = s[cIdx];
    tree[idx].parent = nIdx;
    tree[idx].child.clear();
    tree[nIdx].child[s[cIdx]] = idx;
  }
  if(cIdx == len-1)
    tree[tree[nIdx].child[s[cIdx]]].isWord=1;
  else insert(tree[nIdx].child[s[cIdx]],cIdx+1);
}
void traverse(int x, int depth){
  depth += tree[x].isWord;
  if(ma < depth)ma = depth, nn = x;
  for(auto i: tree[x].child)traverse(i.se,depth);
}
int main(){
  idx = 0;  tree[0].kar=0;  ma=-1;
  while(scanf("%s",s)){
    if(strcmp(s,".") == 0)break;
    len = strlen(s);
    insert(0,0);
  }
  nn = 0;
  traverse(0,0);
  int res[100], dd = 0;  char ss[100];
  while(nn){
    ss[dd] = tree[nn].kar;
    res[dd++] = tree[nn].isWord;
    nn = tree[nn].parent;
  }
  reverse(ss,ss+dd);  reverse(res,res+dd);  ss[dd]=0;
  FOR(i,dd) if(res[i]){
    char temp = ss[i+1];  ss[i+1] = 0;
    printf("%s\n",ss);  ss[i+1] = temp;
} }
Input  : i if in input int integer output
Output : i in int integer

// Another trie
#define MA 5000010
typedef struct Tnode{
  int counter;  char p;  int child[3];
} node;
char s[210];  node trie[MA];  int idx,len,res;

void insert(int trid, int id){
  trie[trid].counter++;
  if(id == len)return;
  if(trie[trid].child[s[id]-'0']==-1){
    trie[++idx].p = s[id];
    trie[trid].child[s[id]-'0'] = idx;
  }
  insert(trie[trid].child[s[id]-'0'],id+1);
}
void calc(int idx, int length){
  res = max(res,trie[idx].counter * length);
  if(trie[idx].child[0] != -1)
    calc(trie[idx].child[0],length+1);
  if(trie[idx].child[1] != -1)
    calc(trie[idx].child[1],length+1);
}
Int main(){
  int t,n;
  scanf("%d",&t);
  while(t--){
    idx = 0;
    scanf("%d",&n);
    FOR(i,MA){
      trie[i].counter = 0;
      trie[i].child[0] = trie[i].child[1]=-1;
    }
    while(n--){
      scanf("%s",s);  len = strlen(s);  insert(0,0);
    }
    res = 0;  calc(0,0);  printf("%d\n",res);
} }
```

MO ALGORITHM

```
typedef pair<pii,int> iii;
int blocks,n,l,r,q,ans;
int answer[300100],cnt[1000100],a[300100];
vector<iii> v;

bool cmp(iii x, iii y){
  if(x.fi.fi/blocks == y.fi.fi/blocks)
    return x.fi.se<y.fi.se;
  return x.fi.fi/blocks < y.fi.fi/blocks;
}
void add(int position){
  if(++cnt[a[position]]==1)ans++;
}
void remove(int position){
  if(--cnt[a[position]]==0)ans--;
}
int main(){
  scanf("%d",&n);
  blocks =(int)sqrt(n);
  FOR(i,n)scanf("%d",&a[i]);
  scanf("%d",&q);
  FOR(i,q){
    scanf("%d%d",&l,&r); l--; r--;
    v.pb(mp(mp(l,r),i));
  }
  sort(v.begin(),v.end(),cmp);
  int currentL = 0, currentR = -1;
  FOR(i,(int)v.size()){
    l = v[i].fi.fi; r = v[i].fi.se;
    int idx = v[i].se;
    while(currentR < r)add(++currentR);
    while(currentR > r)remove(currentR--);
    while(currentL < l)remove(currentL++);
    while(currentL > l)add(--currentL);
    answer[idx]= ans;
  }
  FOR(i,q)printf("%d\n",answer[i]);
}
```

MISOF TREE

```
class KthTree{
  private:
    int**tree, nlv;  int lv(int);
  public:
    KthTree(int);
    ~KthTree();
    void insert(int);
    void erase(int);
    int KthElement(int);
};
int KthTree::lv(int n, int ret = 0){
  while((1<<ret)<=n)ret++;  return ret;
}
KthTree::KthTree(int n){
  this->nlv = this->lv(n) + 1;
  tree=(int**)malloc(sizeof(int*)*(this->nlv));
  FOR(i,this->nlv){
tree[i]=(int*)malloc(sizeof(int)*(1<<(this->nlv-1)));
memset(tree[i],0,sizeof(int)*(1<<(this->nlv-1)));
  }
}
```

```
KthTree::~KthTree(){
  FOR(i,this->nlv)free(tree[i]);  free(tree);
}
void KthTree::insert(int n){
  FOR(i,this->nlv)tree[i][n]++, n >>= 1;
}
void KthTree::erase(int n){
  FOR(i,this->nlv)tree[i][n]--, n >>= 1;
}
int KthTree::KthElement(int k){
  int a =0, b = (this->nlv)-1;
  while(b--) if(tree[b][a<<=1]<k)k -= tree[b][a++];
  return a;
}
```

HEAVY LIGHT DECOMPOSITION

```
class Tree{
  private:
    int num, idx, n, elder;
    int dat[50005<<2];
    //LCA parent
    int par[50005][25];
    int depth[50005];
    int hashh[50005];
    int chain[50005];
    int headd[50005];
    vi child[50005];
    int count[50005];//count subtree
    //generate function
    void genDepth(int,int);
    void genHLD(int);
    //LCA func
    int getNPar(int,int);
    int getAnc(int,int);
    //segment tree RURQ
    void upd(int,int,int,int,int,int);
    int que(int,int,int,int);
  public:
    void clear(int);
    void push(int,int);
    void generate();
    void upd(int,int,int);
    int que(int);
};
void Tree::genDepth(int _node, int _depth=0){
  depth[_node] = _depth;  count[_node] = 1;
  for(auto x: child[_node]){
    if(x == par[_node][0])continue;
    par[x][0] = _node;
    genDepth(x,_depth+1);
    count[_node] += count[x];
} }
void Tree::genHLD(int _node){
  hashh[_node] = ++idx;
  chain[_node] = num;
  pii temp = mp(-1,-1);
  for(auto x: child[_node])
    if(count[x]>temp.se && x!=par[_node][0])
      temp=mp(x,count[x]);
  if(temp.fi != -1)genHLD(temp.fi);
  for(auto x: child[_node]){
    if(x==temp.fi || x==par[_node][0])continue;
    chain[x]=++num;  headd[num]=x;  genHLD(x);
} }
int Tree::getNPar(int _node, int _n){
  for(int j=0; j<15 && _n; j++, _n>>=1)
    if(_n&1)_node = par[_node][j];
  return _node;
}
int Tree::getAnc(int _u, int _v){
  //stabilize depth
  if(depth[_u]< depth[_v])
    _v = getNPar(_v,depth[_v]-depth[_u]);
  if(depth[_u]>depth[_v])
    _u = getNPar(_u,depth[_u]-depth[_v]);
  //u==v
  if(_u == _v)return _u;
  //search LCA
  FORD(j,20)
    if(depth[_u]>=(1<<j)&&par[_u][j]!=par[_v][j])
      _u = par[_u][j]; _v=par[_v][j];
  return par[_u][0];
}
Void ::upd(int idx, int a,int b, int l,int r,int val){
  //base case
  if(b<l||r<a)return;
  if(l<=a&&b<=r){ dat[idx]+=val;  return;}
  //reccurence
  int mid = (a+b)>>1;
  upd(2*idx,a,mid,l,r,val);
  upd(2*idx+1,mid+1,b,l,r,val);
}
int Tree::que(int idx, int a, int b, int pos){
  //base case
  if(b<pos || pos<a)return 0;
  if(pos<=a && b<=pos)return dat[idx];
  //reccurence
  int mid = (a+b)>>1;
  if(pos <= mid)return dat[idx]+que(2*idx,a,mid,pos);
  return dat[idx]+que(2*idx+1,mid+1,b,pos);
}
//public
void Tree::clear(int _n){
  n = _n;
  FOR(i,_n+3){
    FOR(j,23)par[i][j] = 0;
    child[i].clear();
    hashh[i] = depth[i] = count[i] = 0;
    chain[i] = headd[i] = -1;
  }
  FOR(i,_n<<2)dat[i] = 0;
  num = idx = elder = 0;
}
void Tree::push(int _u, int _v){
  child[_u].pb(_v);  child[_v].pb(_u);
}
void Tree::generate(){
  //generate LCA
  genDepth(elder);
  REP(j,1,20) REP(i,0,n-1)
    par[i][j] = par[ par[i][j-1]][j-1];
  //generate HLD :)
  chain[elder] = ++num;  headd[num] = elder;
  genHLD(elder);
}
void Tree::upd(int _u, int _v, int _val){
  int anc = getAnc(_u,_v);
  while(1){ // run from U
    int fron = headd[chain[_u]];
    if(chain[anc] == chain[_u])fron=anc;
    upd(1,1,n,hashh[fron],hashh[_u],_val);
    if(chain[anc] == chain[_u])break;
    _u = par[fron][0];
  }
  while(1){ // run from V
    int fron = headd[chain[_v]];
    if(chain[anc] == chain[_v])fron=anc;
    upd(1,1,n,hashh[fron],hashh[_v],_val);
    if(chain[anc] == chain[_v])break;
    _v = par[fron][0];
  }
  //substract anc
  upd(1,1,n,hashh[anc],hashh[anc],-_val);
}
int Tree::que(int pos){
  return que(1,1,n,hashh[pos]);
}
```

LIS (LONGEST INCREASING SUBSEQUENCE)

```
vi indi, arr((int)1e6,0);  arr[0]=INF;  int cnt=0;

FOR(i,n){
  int lo = 0, hi = cnt, mid;
  while(lo<=hi){
    mid = (lo+hi)/2;
    if(arr[mid] >= lst[i])hi = mid-1;
    else lo = mid+1;
  }
  arr[lo] = lst[i];  indi.pb(lo);  if(cnt<lo)cnt=lo;
}
vi ans;  int idx = n-1;
while(cnt>0){
```

```
  if(indi[idx]==cnt)ans.pb(lst[idx]), cnt--;
  idx--;
}
reverse(ans.begin(),ans.end());
```

CYCLIC LCS (LONGEST COMMON SUBSEQUENCE)

```
// O(nm)    https://arxiv.org/pdf/1208.0396v3.pdf
const int N = 4000+10;
int dp[N][N], from[N][N];
int eq(int a, int b){ return s[(a-1)%n]==t[(b-1)%m]; }
int clcs(){
  REP(i,0,n<<1) REP(j,0,m)dp[i][j] = from[i][j] = 0;
  REP(i,0,n<<1) REP(j,0,m){
    dp[i][j] = 0;
    if(j && dp[i][j-1] > dp[i][j])
      dp[i][j] = dp[i][j -1], from[i][j]=0;
    if(i&&j && eq(i,j) && dp[i-1][j-1]+1 > dp[i][j])
      dp[i][j] = dp[i-1][j-1]+1, from[i][j]=1;
    if(i && dp[i-1][j] > dp[i][j])
      dp[i][j] = dp[i-1][j], from[i][j]=2;
  }
  int ret = 0;
  FOR(i,n){
    ret = max(ret,dp[i+n][m]);
    int x = i+1, y = 0;
    while(y<=m && from[x][y]==0) y++;
    for(; y<=m && x<=n*2; x++){
      from[x][y] = 0;  dp[x][m]--;
      if(x == n*2)break;
      for(; y <= m; y++){
        if(from[x+1][y] == 2)break;
        if(y+1<= m && from[x+1][y+1]==1){
          y++;  break;
  } } } }
  return ret<<1;
}
```

LCIS (LONGEST COMMON INCREASING SUBSEQUENCE)

```
const int OO =(int)1e9;  const ll INF =(ll)1e18;
int getint(){ int x;  scanf("%d",&x);  return x; }
const int MAXN =512;
int A[MAXN];  int na;  int B[MAXN];  int nb;
int prev[MAXN][MAXN];
ll best[MAXN][MAXN];
int pos[MAXN][MAXN];

void printPath(int path){
  if(prev[path>>10][path&1023] == -1)
    printf("%d",A[path>>10]);
  else{
    printPath(prev[path>>10][path&1023]);
    printf(" %d",A[path>>10]);
} }
int main(){
  FOR(i,na=getint())A[i] = getint();
  FOR(i,nb=getint())B[i] = getint();
  FOR(i,nb) FOR(j,nb)best[i][j] = INF;
  FOR(i,na) FOR(j,nb)prev[i][j] = -1;
  FOR(i,na){
    int x = -1, p =0;
    FOR(j,nb){
      if(A[i] == B[j]){
        int y = p;  while(best[j][y]<A[i])y++;
        best[j][y] = A[i];  pos[j][y] = (i<<10)+j;
        if(y >0)prev[i][j] = pos[j][y-1];
        x = p = y;
      } else if(x!=-1 && best[j-1][x]<best[j][x]){
        best[j][x] = best[j-1][x];
        pos[j][x] = pos[j-1][x];
      } else x = -1;
  } }
  int len = -1;
  FORD(i,nb) if(best[nb-1][i]!=INF){ len=i;  break; }
  printf("%d\n",len+1);
  if(len >=0)printPath(pos[nb-1][len]);  puts("");
}
```

DP Buka String & Lipat String

```
struct tipetipe{ int min, nkurung; };  string s2;
tipetipe memo[305][305];  int n, path[305][305][4];

string buka(int& i, string s){
  int len = s.length(), angka=0;  string tmp;
  while(i<len && s[i]!=')'){
    if(s[i]>='0'&& s[i]<='9')
      angka = angka*10+(s[i]-'0');
    else if(s[i]=='('){
      string tmp2 = buka(++i,s);
      REP(i,angka)tmp += tmp2;
      angka = 0;
    } else tmp = tmp+s[i];
    i++;
  }
  return tmp;
}
tipetipe hohoho(int l, int r){
  // printf("%d %d\n",l,r);  LOL;  STOP;
  if(memo[r][l].min != -1)return memo[r][l];
  tipetipe ret;  ret.min = r-l+1;  ret.nkurung = 0;
  path[l][r][0] = 0;
  REP(i,1,(r-l+1)/2) if((r-l+1)%i == 0){
    string t1 = s2.substr(0+l,i);
    string t2 = s2.substr(i+l,i);
    int ni = 1, j = i+l;
    while(t1==t2 && j<=r){
      ni++;  j+=i;  t2 = s2.substr(j,i);
    }
    if(ni == (r-l+1)/i){
      int dig = 0, nii = ni;
      while(ni>0)dig++, ni /= 10;
      tipetipe tmp = hohoho(l,l+i-1);
      tmp.min += dig+2;  tmp.nkurung += 2;
      if(tmp.min < ret.min){
        ret = tmp;
        path[l][r][0] = 1;
        path[l][r][1] = nii;
        path[l][r][2] = i;
      } else if(tmp.min==ret.min && tmp.nkurung<  \
      ret.nkurung){
        ret = tmp;  path[l][r][0]=1;
        path[l][r][1]= nii;  path[l][r][2]= i;
  } } }
  REP(i,l,r-1){
    tipetipe tmp, left, right;
    left = hohoho(l,i);  right = hohoho(i+1,r);
    tmp.min = left.min + right.min;
    tmp.nkurung = left.nkurung + right.nkurung;
    if(tmp.min< ret.min){
      ret = tmp;  path[l][r][0]=2;  path[l][r][1]=i;
    } else if(tmp.min== ret.min&& tmp.nkurung <  \
    ret.nkurung){
      ret = tmp;  path[l][r][0]=2;  path[l][r][1]=i;
    }
  }
  //printf("l->%d  r->%d  res->%d  path->%d\n", \
  l,r,ret,path[l][r][0] );
  return memo[r][l]= ret;
}
string search_path(int l, int r){
  string ret;
  if(path[l][r][0]==0) FOR(i,l,r)ret += s2[i];
  if(path[l][r][0]==1){
    char tmpx[10];
    sprintf(tmpx,"%d",path[l][r][1]);  ret += tmpx;
    ret+="("+search_path(l,l+path[l][r][2]-1)+")";
  }
  if(path[l][r][0]==2){
    ret += search_path(l,path[l][r][1]);
    ret += search_path(path[l][r][1]+1,r);
  }
  return ret;
}
int main(){
  cin >> n;
  FOR(i,n){
    string s1;  cin >> s1;
    int z = 0;  s2 = buka(z,s1);
    memset(memo,-1,sizeof(memo));
    hohoho(0,s2.length()-1);
    cout << s2 << endl;
    cout << search_path(0,s2.length()-1) << endl;
} }
```

```
// input
2(haue)yueuyeuue3(xxzz)zzxx
hauehaueyueuyeuuexxzzxxzzxxzzzzxx
2(haue)yueuyeuue3(xxzz)zzxx
```

MATRIX CHAIN MULTIPLICATION

```
const int MAX=100;  const int INF=(int)2e9;
int p[MAX+1], m[MAX+1][MAX+1], s[MAX+1][MAX+1];

void MCM(int n){
  FOR(i,n+1)m[i][i]=0;
  REP(l,2,n)REP(i,1,n-l+1){
    Int j = i+l-1;   m[i][j] = INF;
    REP(k,i,j-1){
      int q = m[i][k]+m[k+1][j]+p[i-1]*p[k]*p[j];
      if(q < m[i][j])m[i][j] = q, s[i][j] = k;
} } }
void print_MCM(int i, int j){
  if(i == j)printf("A%d",i);
  else{
    printf("(");  print_MCM(i,s[i][j]);
    printf(" x ");  print_MCM(s[i][j]+1,j);
    printf(")");
} }
int main(){
  int n, i, j, cs = 1;
  while(scanf("%d",&n) == 1){
    if(n == 0)break;
    FOR(i,n)scanf("%d%d",&p[i],&p[i+1]);
    printf("Case %d: ",cs++);  MCM(n);
    print_MCM(1,n);  puts("");
} }
```

MIN CUT POLYGON TRIAGULATION

```
int getCost(pii a, pii b, pii c){
  return getDist2(a,b) +getDist2(b,c) +getDist2(c,a);
}
int minCutPolygonTriagulation(){
  int res=0, dp[105][105];
  FOR(i,103) FOR(j,103)dp[i][j] = 0;
  FOR(i,n)res += getDist2(dat[i],dat[(i+1)%n]);
  //DP
  FOR(xx,n) for(int i=0,j=xx; j<n; i++,j++){
    dp[i][j] = (j<i+2) ? 0 : INF;
    REP(k,i+1,j-1)dp[i][j]=min(dp[i][j],dp[i][k]    \
      +dp[k][j]+getCost(dat[i],dat[j],dat[k]));
  }
  return(dp[0][n-1]-res)>>1;
}
```

KADANE

```
// Kadane 1D
k = 0;  l = 0;  s = -INF;  t = 0;  j = 1;
REP(i,1,n){
  t += a[i];
  if(t > s)k = j, l = i, s = t;
  if(t < 0)t = 0, j = i+1;
}
// Kadane 2D
int N, t =0, a[100][100], pr[100];
int S = 1<<31, s=0, k, l, x1=0,x2=0, y1=0,y2=0, j;

FOR(z,N){
  FOR(i,N)pr[i] = 0;
  REP(i,z,N-1){
    t = j = k = l = 0, s =1<<31;
    FOR(i,n){
      pr[i] = pr[i]+ a[x][i];
      t = t + pr[i];
      if(t > s)s = t, k = i, l = j;
      if(t < 0)t = 0, j = i+1;
    }
    if(s > S)S = s, x1 = x, y1 = k, x2 = z, y2 = l;
} }
cout << x1 <<" "<< y1 <<" "<< x2 <<" "<< y2 <<endl;
cout << S;
```

DP SOS

```
// Given array A[] of 2^N integers, find sum of all
// F(x), where F(x) = sum of all A[i] where x&i=i

// O(3^N)  This one can be used for various other
// problems. Flexible;  iterate over all the masks
FOR(mask,(1<<n)){
  F[mask] = A[0];
  // iterate over all the subsets of the mask
  for(int i=mask; i>0; i=(i-1)&mask)F[mask] += A[i];
}
// O(N * 2^N)  memory optimized, super easy to code.
FOR(i,(1<<N))F[i] = A[i];
FOR(i,N) FOR(mask,(1<<N)) if(mask&(1<<i))
  F[mask] += F[mask^(1<<i)];
```

DP CONVEX HULL

```
// Name        : Convex Hull Optimization 1
// Recurrence  : dp[i] = min(dp[j] + b[j]*a[i])
// Condition   : b[j] >= b[j+1] and a[i] <= a[i+1]
// Complexity  : O(n^2) -> O(n)

// Name        : Convex Hull Optimization 2
// Recurrence  : dp[i][j] = min(dp[i-1][k] + b[k]*a[j])
// Condition   : b[k] >= b[k+1] and a[j] <= a[j+1]
// Complexity  : O(kn^2) -> O(kn)
struct line{
  ll a,b;  ll getval(ll x){ return a*x+b; }
};
ll xsmall(line l1, line l2, line l3){
  ll x1 = (l3.b-l1.b)*(l1.a-l2.a);
  ll x2 = (l2.b-l1.b)*(l1.a-l3.a);
  return x1<x2;
}
void getpoint(int k){
  v.clear();
  REP(i,1,n){
    line temp;  temp.a = -i;
    temp.b = (i*s[i]) - sum[i] + dp[i][k-1];
    while(v.size()>=2 && xsmall(v[v.size()-2],      \
      v.back(),temp))v.pob();
    v.pb(temp);
} }
int main(){
  scanf("%d%d",&n,&k);
  REP(i,1,n){
    scanf("%lld",&f[i]);
    sum[i] = sum[i-1]+(f[i]*(i-1));
    s[i] = s[i-1]+f[i];
  }
  REP(i,1,n)dp[i][1] = sum[i];  int point = 0;
  REP(i,2,k){
    getpoint(i);  point = 0;
    REP(j,1,n) while(point<v.size()-1 && v[point]   \
      .getval(s[j])>v[point+1].getval(s[j]))point++;
    dp[j][i] = sum[j]+v[point].getval(s[j]);
  }
  printf("%lld\n",dp[n][k]);
}
```

DP DIVIDE AND CONQUER

```
// dp[i][j] = min(dp[i-1][k] + C[k][j]) for (k<j)
// A[i][j] <= A[i][j+1]
// O(knlogn)

void solve(int k, int l, int r, int L , int R){
    if(l > r)return;
    int mid = (l+r)/2, opt = -1;
    REP(i,L,min(R,mid)){
        ll tmp = dp[k-1][i]+cost[i+1][mid];
        if(tmp < dp[k][mid])dp[k][mid] = tmp, opt = i;
    }
    solve(k,l,mid-1,L,opt);  solve(k,mid+1,r,opt,R);
}
int main(){
    // init array of dp to infinity
    // init array dp[1][i] = to cost[1][i]
    REP(i,2,k)solve(i,1,n,1,n);
}
```

DP KNUTH YAO

```
// SPOJ BREAKING STRINGS
// dp[i][j] = min(dp[i][k]+dp[k][j])+C[i][j]
// for (i<k<j)   and   A[i,j-1] <= A[i,j] <= A[i+1,j]
// O(n^2)
ll n, dp[1005][1005], f[1005]; int m, mid[1005][1005];

int main(){
```

```
  while(scanf("%lld%d",&n,&m) == 2){
    REP(i,1,m)scanf("%lld",&f[i]);
    f[0] = 0;  f[m+1] = n;
    FOR(i,m+2) FOR(j,m+2)dp[i][j]=1e18, mid[i][j]=i;
    FOR(s,m+2) FOR(l,m-s+2){
      int r = l+s;
      if(s < 2){
        dp[l][r] = 0;  mid[l][r] = l;  continue;
      }
      int mleft = mid[l][r-1], mright = mid[l+1][r];
      dp[l][r] = 1e18;
      REP(i,mleft,mright){
        ll tmp = dp[l][i]+dp[i][r]+f[r]-f[l];
        if(tmp < dp[l][r])dp[l][r]=tmp, mid[l][r]=i;
    } }
    printf("%lld\n",dp[0][m+1]);
} }
```

TOPOLOGICAL SORT

```
void topoSort(int v){
  vis[v] = 1;
  FOR(i,(int)ad[v].size())
    if(!vis[ad[v][i]])topoSort(ad[v][i]);
  topoSorted.pb(v);
}
int main(){
  topoSorted.clear();
  REP(i,N) if(!vis[i])topoSort(i);
  reverse(topoSorted.begin(),topoSorted.end());
}
```

Kahn's Algorithm

```
void topoSortKahn(){
  // get all indegrees by traverse adj lists O(V+E)
  vi in_degree(V,0);
  FOR(u,V) for(auto x: adj[u])in_degree[x]++;
  queue<int> q;
  // queue all indegree == 0
  FOR(i,v) if(in_degree[i] == 0)q.push(i);
  int cnt = 0;  vi res;
  while(!q.empty()){
    int u = q.front();  q.pop();
    res.pb(u);
    for(auto x: adj[u])
      if(--in_degree[x]==0)q.push(x);
    cnt++;
  }
  // Check if there was a cycle
  if(cnt != V){ puts("Cycle detected!");  return; }
  for(auto x: top_order)printf("%d ",x);  puts("");
}
```

K-th SHORTEST PATH

```
const int MAXN = 100;  const int MAXK = 40;
vii g[MAXN];  priority_queue<pii>pq;
int visit[MAXN], n, k;

int solve(int s, int t){
  FILL(visit,0);  pq.push(MP(0,s));  int ret = 0;
  while(!pq.empty()){
    int awal = -pq.top().fi;
    int f = pq.top().se; pq.pop();
    if(visit[f]>=k)continue;
    visit[f]++;
    if(f==t)ret+=awal;
    for(auto x: g[f]) if(visit[x.fi]<k)
      pq.push(MP(-(awal+x.se),x.fi));
  }
  return ret;
}
int main(){
  k = GI;  n = GI;
  FOR(nedge,GI){
    int a = GI-1, b = GI-1, c = GI;
    g[a].pb(mp(b,c));  g[b].pb(mp(a,c));
  }
  printf("%d\n",solve(0,n-1));
}
// k = shortest path ke berapa.
// n = jumlah vertex.
// solve(i,j); shortest path dari i ke j.
```

MANHATTAN SPANNING TREE

```
const int INF =(int)1e9;
struct Pos{ int x,y, diff,sum, id; } p[150000];
vi BIT;  vector<pair<pair<int,int>,int>> Raw_Edges;

bool sortpoint(Pos A, Pos B){
  if(A.x != B.x)return A.x > B.x;
  return A.y > B.y;
}
int GetNearest(int x){
  int ret =-1;
  for(; x>0; x-=(x&(-x)))
    if(BIT[x]!=-1&&(ret==-1 || p[BIT[x]].sum <      \
    p[ret].sum))ret = BIT[x];
  return ret;
}
void Insert(int x,int id){
  int maxi = BIT.size();
  for(; x<maxi; x+=(x&(-x)))
    if(BIT[x]==-1 || (p[id].sum<p[BIT[x]].sum))
      BIT[x] = id;
}
void AddEdges(){
  viaxis;  axis.pb(-INF);
  FOR(a,n)axis.pb(p[a].diff);
  sort(axis.begin(),axis.end());
  axis.erase(unique(axis.begin(),axis.end()),       \
    axis.end());
  sort(p,p+n,sortpoint);
  BIT = vi(axis.size(),-1);
  FOR(a,n){
    int pos = lower_bound(axis.begin(),axis.end(),  \
      p[a].diff) - axis.begin();
    int nearest = GetNearest(pos);
    if(nearest != -1)Raw_Edges.pb(mp(mp(p[a].id,    \
      p[nearest].id),p[nearest].sum - p[a].sum));
    Insert(pos,a);
} }
Int main(){
  FOR(a,n){
    scanf("%d%d",&p[a].x,&p[a].y);
    p[a].sum = p[a].x + p[a].y;
    p[a].diff = p[a].x - p[a].y;  p[a].id = a;
  }
  FOR(a,4){
    FOR(b,n){
      if(a==1 || a==3){
        swap(p[b].x,p[b].y);
        p[b].diff = p[b].x - p[b].y;
      } else if(a==2){
        p[b].x =-p[b].x;
        p[b].sum = p[b].x + p[b].y;
        p[b].diff = p[b].x - p[b].y;
      }
    }  AddEdges();
  }
  //DO MST
}
```

DIRECTED MST

```
int con[MAXN][MAXN], in[MAXN];
int vis[MAXN], active[MAXN], n;
//vis state: 0:unvisited, 1:visited, 2:cycle, 3:active
int cycle(int idx, int& res){
  if(vis[idx] == 0){
    vis[idx] = 3;
    vis[idx] = (in[idx]!=-1) ? cycle(in[idx],res):1;
  } else if(vis[idx] == 3){
    set<int> pseudoNodes;  int minCycle = INF;
    for(int j=idx; ; j=in[j]){
      pseudoNodes.insert(j);
      minCycle <?= con[in[j]][j];
      if(in[j]==idx)break;
    }
    int minCost = INF, bestParent;
    for(auto j: pseudoNodes){
      assert(active[j]);
      FOR(i,n) if(active[i] && con[i][j]!=-1 &&     \
      !pseudoNodes.count(i)){
        if(minCost > con[i][j]-con[in[j]][j]){
          minCost = con[i][j]-con[in[j]][j];
          bestParent = i;
```

```
        }
        con[i][j] -= con[in[j]][j]-minCycle;
    } }
    res += minCost;
    in[idx] = bestParent;
    for(auto j: pseudoNodes){
      FOR(i,n) if(active[i] && i!=idx &&          \
        con[i][j]!=-1) if(con[i][idx]==-1 ||      \
        con[i][idx]>con[i][j])
          con[i][idx] = con[i][j];
      FOR(i,n) if(active[i]&&i!=idx&&con[j][i]!=-1){
        if(in[i]==j)in[i] = idx;
        if(con[idx][i]==-1 || con[idx][i]>con[j][i])
          con[idx][i]= con[j][i];
      }
      if(j != idx)active[j] = false;
    }
    vis[idx] = 2;
  }
  return vis[idx];
}
int dmst(int root){
  int res = 0;  in[root] = -1;
  FOR(i,n)con[i][root] = -1;
  FOR(i,n) if(i!=root){
    int minCost = INF;
    FOR(j,n) if(con[j][i]!=-1 && con[j][i]<minCost)
      minCost = con[j][i], in[i] = j;
    res += minCost;
  }
  memset(active,1,sizeof(active));
  while(1){
    memset(vis,0,sizeof(vis));
    bool hasCycle = cycle(root,res)==2;
    FOR(i,n) if(active[i] && !vis[i])
      hasCycle |=(cycle(i,res)==2);
    if(!hasCycle)return res;
} }
int dfs(int idx){
  if(vis[idx])return 0;
  int ret = 1;  vis[idx] = true;
  FOR(i,n) if(con[idx][i] != -1)ret += dfs(i);
  return ret;
}
bool connected(int root){
  memset(vis,0,sizeof(vis));  return dfs(root)==n;
}
int main(){
  int nTC, m, u, v, w;
  scanf("%d",&nTC);
  REP(TC,1,nTC){
    printf("Case #%d: ",TC);
    scanf("%d%d",&n,&m);  memset(con,-1,sizeof(con));
    FOR(i,m)scanf("%d%d%d",&u,&v,&w), con[u][v]= w;
    if(!connected(0))puts("Possums!");
    else printf("%d\n",dmst(0));
} }
```

BELLMAN FORD (Negative Cycle Detection)

```
int bellmanFord(int s){
  int i, j;  FOR(i,n)d[i]= INF;  d[s]=0;
  FOR(i,n-1) FOR(j,e)
    if(d[edge[j].v] > d[edge[j].u]+edge[j].w)
      d[edge[j].v] = d[edge[j].u]+edge[j].w;
  FOR(j,e) if(d[edges[j].u]+edges[j].w<d[edges[j].v])
    return 0;
  return 1;
}
```

DINIC ALGORITHM

```
#define SET(p) memset(p,-1, sizeof(p))
#define CLR(p) memset(p, 0, sizeof(p))
#define i64 long long
const int INF = 0x7fffffff, MAXN = 5005, MAXE = 60006;
int src, snk, nNode, nEdge;
int Q[MAXN], fin[MAXN], pro[MAXN], dist[MAXN];
int flow[MAXE], cap[MAXE], next[MAXE], to[MAXE];

void init(int _src,int _snk,int _n){
  src=_src, snk=_snk, nNode=_n, nEdge=0;  SET(fin);
}
void add(int u,int v,int c){
  to[nEdge] = v, cap[nEdge] = c, flow[nEdge] = 0;
  next[nEdge] = fin[u], fin[u] = nEdge++;
  to[nEdge] = u, cap[nEdge] = c, flow[nEdge] = 0;
  next[nEdge] = fin[v], fin[v] = nEdge++;
}
bool bfs(){
  int st, en, i, u, v;  SET(dist);
  dist[src] = st = en =0;  Q[en++]= src;
  while(st < en){
    u = Q[st++];
    for(i=fin[u]; i>=0; i=next[i]){
      v = to[i];
      if(flow[i]< cap[i]&& dist[v]==-1)
        dist[v]= dist[u]+1, Q[en++]= v;
  } }
  return dist[snk]!=-1;
}
int dfs(int u,int fl){
  if(u==snk)return fl;
  for(int& e = pro[u], v, df; e>=0; e = next[e]){
    v = to[e];
    if(flow[e]<cap[e] && dist[v]==dist[u]+1){
      df = dfs(v,min(cap[e]-flow[e],fl));
      if(df>0)flow[e]+= df,flow[e^1]-= df, return df;
    }
  } return 0;
}
i64 dinitz(){
  i64 ret = 0, df;
  while(bfs()){
    REP(i,1,nNode)pro[i]= fin[i];
    while(true){
      df = dfs(src,INF);
      if(df)ret+=(i64)df;
      else break;
  } }
  return ret;
}
int main(){
  int n, e, u, v, c, i;
  scanf("%d%d",&n,&e);  init(1,n,n);
  FOR(i,e){
    scanf("%d%d%d",&u,&v,&c);
    if(u!=v)add(u,v,c);
  }
  printf("%lld\n",dinitz());
}
```

LOWEST COMMON ANCESTOR (SEG TREE)

```
const int MAXN = 200005;  const int INF =(int)1e9;
int n, q, euler[2*MAXN], level[2*MAXN], h[2*MAXN], id;
// h[i] = index of the first occurence of node i in E
vi adj[maxn];bool vis[maxn];
struct node{int level,id;} st[4*2*MAXN];

void eulerTour(int u, int p, int l){
  euler[id] = u;  level[id] = l;
  if(!vis[u])vis[u] = 1, h[u] = id;
  id++;
  for(auto v: adj[u]) if(v != p){
    eulerTour(v, u, l+1);
    euler[id] = u;  level[id++] = l;
} }
void build(int p, int l, int r){
  if(l == r){
    st[p].level = level[l];  st[p].id = l;  return;
  }
  int mid = (l+r)/2;
  build(p*2, l, mid);  build(p*2+1, mid+1, r);
  st[p].level = min(st[p*2].level,st[p*2+1].level);
  st[p].id = (st[p*2].level < st[p*2+1].level) ?    \
    st[p*2].id : st[p*2+1].id;
}
node query(int p,int l,int r, inti,int j){
  if(l > j || r <i){
    node ret;  ret.level = INF;  return ret;
  }
  if(l >= I && r <= j)return st[p];
  int mid = (l+r)/2;
  node ret1 = query(p*2,l,mid,i,j);
```

```
  node ret2 = query(p*2+1,mid+1,r,i,j);
  return (ret1.level < ret2.level) ? ret1 : ret2;
}
int lca(int r,int u,int v){
  int a = min(h[u],h[v]), b = max(h[u],h[v]);
  int lca_id = query(1,1,2*n-1,a,b).id;
  return euler[lca_id];
}
int main(){
  int r, u, v;
  scanf("%d",&n);
  FOR(i,n-1){
    scanf("%d%d",&u,&v);  adj[u].pb(v); adj[v].pb(u);
  }
  memset(euler,0,sizeof(euler));
  memset(level,0,sizeof(level));
  memset(h,0,sizeof(h));  memset(vis,0,sizeof(vis));
  id =1;  eulertour(1,1,0);  memset(st,0,sizeof(st));
  build(1,1,2*n-1);
  scanf("%d",&q);
  while(q--){
    scanf("%d%d%d",&r,&u,&v);
    int lca1, lca2, lca3, ans;
    lca1 = lca(1,r,u);
    lca2 = lca(1,r,v);
    lca3 = lca(1,u,v);
    if(lca1 == lca2 && lca2 == lca3)ans = lca1;
    elseif(lca1==lca2 && lca3!=lca1)ans = lca3;
    elseif(lca1==lca3 && lca2!=lca1)ans = lca2;
    elseif(lca2==lca3 && lca1!=lca2)ans = lca1;
    printf("%d\n",ans);
} }
```

Let LCA(r,u,v) be the LCA of u and v with respect to root r. To compute LCA(r,u,v), we can compute, for any fixed ro (usually let ro = 1),

```
LCA(ro,r,u);  // LCA(1,r,u);
LCA(ro,r,v);  // LCA(1,r,v);
LCA(ro,u,v);  // LCA(1,u,v);
```

and take the "odd man out", i.e., if two are equal and the third are different, then take the third, else if they are all equal, take any one.

MIN COST MAX FLOW

```
int tc, R, C, flow, mf, s, t, costperflow, p[maxn];
int res[maxn][maxn], cost[maxn][maxn], dist[maxn];
vi adj[maxn];

bool bellman(){
  REP(i,s,t)dist[i]=INF, p[i]=-1;
  dist[s]=0;
  queue<int> q; q.push(s);
  while(!q.empty()){
    int u = q.front();  q.pop();
    FOR(i,adj[u].size()){
      int v = adj[u][i], w = cost[u][v];
      if(dist[v]> dist[u]+w && res[u][v]>0){
        p[v] = u;  dist[v] = dist[u]+w;  q.push(v);
  } } }
  return dist[t]!= INF;
}
void augment(int u,int minedge){
  if(s == u){ flow = minedge;return;}
  else if(p[u]!=-1){
    augment(p[u],min(minedge,res[p[u]][u]));
    res[p[u]][u] -= flow;
    res[u][p[u]] += flow;
    costperflow += cost[p[u]][u];
} }
int main(){
  scanf("%d",&tc);
  REP(caseno,1,tc){
    scanf("%d%d",&R,&C);
    memset(res,0,sizeof res);
    memset(cost,0,sizeof cost);
    FOR(i,maxn)adj[i].clear();
    s =0;  t =2*R*C+1;
    REP(i,1,R*C){ adj[s].pb(i);  res[s][i]=1; }
    REP(i,R*C+1,2*R*C){ adj[i].pb(t);  res[i][t]=1; }
    char input[30];
    FOR(i,R){
      scanf("%s",input);
      FOR(j,C){
        int u = id(i,j), d = direction(input[j]);
        FOR(k,4){
          int x = i+dr[k], y = j+dc[k];
          if(x<0 || y<0|| x>=R || y>=C)continue;
          int v = R*C + id(x, y);
          int c = k==d ? 0 : 1;
          adj[u].pb(v);  adj[v].pb(u);
          res[u][v] = 1;  res[v][u] = 0;
          cost[u][v] = c;  cost[v][u] = -c;
    } } }
    int ans = 0; mf = 0;
    while(bellman()){
      flow = 0;  costperflow = 0;
      augment(t,INF);  ans += costperflow;
    }
    printf("Case #%d: %d\n",caseno,ans);
} }
```

MIN COST MAX FLOW (Multiple Edge)

```
typedef struct{ int v, flow, cost, cap; } node;
node parent[MAX_N+1];  vector<node>  adj[MAX_N+1];
int dist[MAX_N+1], from[MAX_N+1], v, e;

int augment(int s,int t){
  bool update;
  REP(i,1,v)dist[i] = INF, from[i] = -1;
  dist[s]=0;
  REP(i,1,v-1){ //bellman ford
    update = false;
    REP(j,1,v) FOR(k,adj[j].size())
    if(adj[j][k].flow < adj[j][k].cap){
      int edgeCost = adj[j][k].cost;
      if(adj[j][k].flow <0)edgeCost = -edgeCost;
      if(dist[adj[j][k].v] > dist[j]+edgeCost){
        dist[adj[j][k].v] = dist[j]+ edgeCost;
        parent[adj[j][k].v] = adj[j][k];
        from[adj[j][k].v] = j;
        update = true;
    } }
    if(!update)break;
  }
  if(dist[t]==INF)return 0;
  int p = t, f = INF, pa;  node temp;
  for(; from[p]!=-1; p=from[p]){
    pa = from[p];  temp = parent[p];
    f = min(f,temp.cap-temp.flow);
  }
  p = t;
  for(; from[p]!=-1; p=from[p]){
    pa = from[p];
    FOR(i,adj[pa].size()){
      if(adj[pa][i].v==p &&adj[pa][i].cost==       \
      parent[p].cost){
        adj[pa][i].flow += f;  break;
    } }
    FOR(i,adj[p].size()){
      if(adj[p][i].v==pa && adj[p][i].cost ==       \
      parent[p].cost){
        adj[p][i].flow-=f;  break;
  } } }
  return f*dist[t];
}
void addedge(int v1,int v2,int c,int cap,int flow=0){
  bool f = true;
  FOR(i,adj[v1].size())
    if(adj[v1][i].v==v2 && adj[v1][i].cost==c){
      adj[v1][i].cap+=cap;  adj[v1][i].flow+=flow;
      f = false;  break;
    }
  if(f)adj[v1].pb((node){v2,flow,c,cap});
}
//Add undirected edge a<->b dengan cost c, kapasitas 1
addedge(a,b,c,1);  addedge(b,a,c,1);
directed edge IN(i)-> OUT(i) dengan cost a dan cap 1
addedge(IN(i),OUT(i),-a,1,0);
addedge(OUT(i),IN(i),a,1,1); //blocking flow
```

MAXFLOW (AdJ List)

```
const int maxv = 100, INF = (int)1e9;
int res[maxv][maxv], s, t, mf, f;  vi p, adj[maxv];
```

```
void augment(int v, int minedge){
  if(v == s){ f = minedge;return;}
  else if(p[v]!=-1){
    augment(p[v],min(minedge, res[p[v]][v]));
    res[p[v]][v]-= f;  res[v][p[v]]+= f;
} }
Int main(){
  mf = 0;
  while(1){ // BFS
    f = 0;  vi dist(maxv,INF); dist[s]=0;
    queue<int> q;  q.push(s);  p.assign(maxv,-1);
    while(!q.empty()){
      int u = q.front();  q.pop();
      if(u == t)break;
      for(auto v: adj[u])
        if(dist[v]==INF && res[u][v]>0)
          dist[v] = dist[u]+1, p[v] = u, q.push(v);
    }
    augment(t,INF);  mf += f;  if(f == 0)break;
  }
  printf("%d\n",mf);
}
```

MIN CUT (Stoer Wagner)

```
int m[600][600], node[600], inset[600], d[600];

int reduce(int n){
  memset(inset,0,sizeof(inset));
  memset(d,0,sizeof(d));
  inset[node[0]] = 1;
  REP(i,1,n-1)d[node[i]] = m[node[0]][node[i]];
  int ltnode = 0;
  REP(_,1,n-1){
    int nd = -1;
    REP(i,1,n-1){
      if(inset[node[i]])continue;
      if(nd ==-1)nd = i;
      if(d[node[i]]> d[node[nd]])nd = i;
    }
    inset[node[nd]] = 1;
    if(_ == n-1){
      FOR(i,n){
        Int xxa = node[ltnode], xxb= node[i];
        m[xxa][xxb] += m[node[nd]][node[i]];
        m[xxb][xxa] = m[xxa][xxb];
      }
      int ret = d[node[nd]];  node[nd] = node[n-1];
      return ret;
    }
    ltnode = nd;
    FOR(i,n){
      if(inset[node[i]])continue;
      d[node[i]]+= m[node[nd]][node[i]];
} } }
int mincut(int n){
  FOR(i,n)node[i]= i;
  int ans = (1<<30);
  while(n > 1){
    int mcut = reduce(n);
    if(mcut < ans)ans = mcut;
    n--;
  }
  return ans;
}
int main(){
  int N, M;
  while(scanf("%d%d",&N,&M)!=EOF){
    memset(m,0,sizeof(m));
    for(int i=0,a,b,c; i<M; i++){
      scanf("%d%d%d",&a,&b,&c);
      m[a][b] += c;  m[b][a] += c;
    }
    printf("%d\n",mincut(N));
} }
```

EULERIAN TOUR CHECKING

```
// Every edge is visited exactly once
int N, M, d[250], di[250];
int m[205][205], pre[205],vis[205];

int flow(int f, int si){
  queue<int> q;  q.push(f);
  memset(vis,0,sizeof(vis));
  memset(pre,-1,sizeof(pre));
  while(!q.empty()){
    int now = q.front();  q.pop();
    if(vis[now] == 2)continue;
    vis[now] = 2;
    if(now == si)break;
    FOR(i,N+1){
      int next = i;
      if(vis[next])continue;
      if(m[now][next]>0)
        vis[next]=1, pre[next]=now, q.push(next);
  } }
  int nt = si, ret = (1<<30);
  while(pre[nt] != -1){
    int bf = pre[nt];
    ret = min(ret,m[bf][nt]);  nt = bf;
  }
  if(ret >= (1<<30))return 0;
  nt = si;
  while(pre[nt] != -1){
    int bf = pre[nt];
    m[bf][nt] -= ret;  m[nt][bf] += ret;  nt = bf;
  }
  return ret;
}
int main(){
  int ntc;
  scanf("%d",&ntc);
  while(ntc--){
    scanf("%d%d",&N,&M);
    memset(d,0,sizeof(d));
    memset(m,0,sizeof(m));
    for(int i=0,a,b,c; i<M; i++){
      scanf("%d%d%d",&a,&b,&c);  d[a]++,d[b]--;
      if(!c)m[a][b]++;
    }
    int can = 1;
    for(int i=1; can && i<=N; i++){
      int x =abs(d[i]);  d[i]>>=1;
      if(x&1)can=0;
    }
    if(can){
      int tflow =0;
      REP(i,1,N){
        if(d[i]<0)m[i][N+1] = -d[i];
        else if(d[i]>0)m[0][i]=d[i], tflow+=d[i];
      }
      int tans, ans = 0;
      while((tans=flow(0,N+1)) > 0)ans += tans;
      can = can && (ans == tflow);
    }
    puts(can ? "possible" : "impossible");
} }
```

MAX BIPARTITE MATCHING

```
vi match, vis;

int aug(int l){
  if(vis[l])return 0;
  vis[l] = 1;
  for(auto r: AdjList[l])
    if(match[r]==-1|| aug(match[r])){
      match[r]= l; return 1;
    }
  return 0;
}
```

MINIMUM VERTEX COVER

```
vi AdjList[100010];  int memo[100010][2];

int solve(int idx, int flag, int prev){
  int ans = 0;
  if(memo[idx][flag] != -1)return memo[idx][flag];
  if(AdjList[idx].size()==1 && AdjList[idx][0]==prev)
    return memo[idx][flag] = flag;
  //reccurence
  if(flag == 0){
    ans = 0;
```

```
    for(auto j: AdjList[idx]) if(prev != j)
      ans += solve(j,1,idx);
  }
  if(flag == 1){
    ans=1;
    for(auto j: AdjList[idx]) if(prev != j)
      ans += min(solve(AdjList[idx][j],1,idx),      \
            solve(AdjList[idx][j],0,idx));
  }
  return memo[idx][flag] = ans;
}
int main(){
  int n,a,b;
  scanf("%d",&n);
  FOR(i,n-1){
    scanf("%d %d",&a,&b);
    AdjList[a].pb(b);  AdjList[b].pb(a);
  }
  memset(memo,-1,sizeof(memo));
  int ans = solve(1,0,-1);
  memset(memo,-1,sizeof(memo));
  int ans2 = solve(1,1,-1);
  printf("%d\n",min(ans,ans2));
}
```

ARTICULATION POINT & BRIDGE

```
vi dfs_num, dfs_low, dfs_parent;
int dfs_counter = 0, rootChildren, dfs_root;

void articulationPoint(int u){
  dfs_num[u] = dfs_low[u] = dfs_counter++;
  for(auto v: AdjList[u]){
    if(dfs_num[v] == INF){
      dfs_parent[v] = u;
      if(u == dfsRoot)rootChildren++;
      articulationPoint(v);
      if(dfs_low[v] >= dfs_num[u]) // Arti. Point
        flag[u] = true;
      if(dfs_low[v] > dfs_num[u]) // bridge Edge(u,v)
        dfs_low[u] = min(dfs_low[u],dfs_low[v]);
    }
    else if(v != parent[u])
      dfs_low[u] = min(dfs_low[u],dfs_num[v]);
} }
//inside int main
dfs_counter = 0;  dfs_num.assign(V,INF);
dfs_low.assign(V,0);  dfs_parent(V,0);
memset(flag,false,sizeof(flag));
FOR(i,V) if(dfs_num[i] == INF){
  dfs_root = i;  rootChildren = 0;
  articulationPoint(i);  flag[i] = (rootChildren>1);
}
```

TARJAN ALGORITHM

```
vi dfs_num, dfs_low, S, visit;

void tarjanSCC(int u){
  dfs_low[u] = dfs_num[u] = dfs_counter++;
  S.push_back(u);  visited[u] = 1;
  for(auto v: AdjList[u]){
    if(dfs_num[v] == UNVISITED)tarjanSCC(v);
    if(visit[v])
      dfs_low[u] = min(dfs_low[u],dfs_low[v]);
  }
  if(dfs_low[u] == dfs_num[u]){
    printf("%SCC %d:",++numSCC);
    while(1){
      int v = S.back();  S.pob();  visit[v]=0;
      printf(" %d",v);  if(u==v)break;
} } }
//inside int main
dfs_num.assign(V,UNVISITED);  dfs_low.assign(V,0);
visit.assigned(V,0);  dfs_counter = numSCC = 0;
FOR(i,V) if(dfs_num[i] == UNVISITED)tarjanSCC(i);
```

HOPCROFT KARP

```
struct graph{
  int L, R;
  vector<vi> adj;
  graph(int L, int R) : L(L), R(R), adj(L+R) {}
  void addEdge(int u, int v){
    adj[u].pb(v+L);  adj[v+L].pb(u);
  }
  int maximum_matching(){
    vi level(L), mate(L+R,-1);

    function<bool(void)> levelize = [&]() { // BFS
      queue<int> Q;
      FOR(u,L){
        level[u] = -1;
        if(mate[u] < 0)level[u] = 0, Q.push(u);
      }
      while(!Q.empty()){
        int u = Q.front();  Q.pop();
        for(int w: adj[u]){
          int v = mate[w];
          if(v < 0)return true;
          if(level[v] < 0){
            level[v] = level[u]+1;
            Q.push(v);
      } } }
      return false;
    };
    function<bool(int)> augment = [&](int u) { // DFS
      for(int w: adj[u]){
        int v = mate[w];
        if(v<0 || (level[v]>level[u]&&augment(v))){
          mate[u] = w;  mate[w] = u;  return true;
      } }
      return false;
    };
    int match = 0;
    while(levelize()) FOR(u,L)
      if (mate[u]<0 && augment(u))match++;
    return match;
} };
int main(){
  int L, R, m, u, v;
  scanf("%d %d %d",&L,&R,&m);  graph g(L,R);
  FOR(i,m)scanf("%d %d",&u,&v), g.addEdge(u,v);
  printf("%d\n",g.maximum_matching());
}
```

EULER PATH + EULER TOUR

```
#define all(c) ((c).begin()), ((c).end())

struct graph{
  int n;
  struct edge{ int src, dst, rev; };
  vector< vector<edge> > adj;
  graph() : n(0) {}
  //graph(int n) : n(n), adj(n) {}
  void addEdge(int src, int dst){
    n = max(n,max(src,dst)+1);
    if(adj.size() < n)adj.resize(n);
    adj[src].pb({src,dst,(int)adj[dst].size()});
    adj[dst].pb({dst,src,(int)adj[src].size()-1});
  }
  // destructive
  vi path;
  void visit(int u){
    while(!adj[u].empty()){
      auto e = adj[u].back();
      adj[u].pob();
      if(e.src >= 0){
        adj[e.dst][e.rev].src = -1;  visit(e.dst);
    } }
    path.pb(u);
  }
  vi eulerianPath(){
    int m = 0, s = -1;
    FOR(u,n){
      m += adj[u].size();
      if(adj[u].size()%2 == 1)s = u;
    }
    path.clear();  if(s >= 0)visit(s);
    if(path.size() != m/2+1)return {};
    return path;
  }
  vi eulerianTour(){
    int m = 0, s = 0;
    FOR(u,n){
      m += adj[u].size();
```

```
    if(adj[u].size() > 0)s = u;
  }
  path.clear();  visit(s);
  if(path.size()!=m/2+1 || path[0]!=path.back())
    return {};
  return path;
} };
int main(){
  int ncase;
  scanf("%d", &ncase);
  REP(icase,1,ncase){
    if(icase > 1)printf("\n");
    printf("Case #%d\n",icase);
    int m,s,t;  graph g;  scanf("%d",&m);
    FOR(i,m)scanf("%d %d",&s,&t), g.addEdge(s-1,t-1);
    auto path = g.eulerianTour();
    if(path.empty())printf("may be lost\n");
    else FOR(i,path.size()-1){
      printf("%d %d\n", path[i]+1, path[i+1]+1);
} } }
```

EDMONDS BLOSSOM

```
// Given a graph G = (V,E), a matching M in G is a set
// of pairwise non-adjacent edges; that is, no two
// edges share a common vertex. A maximum matching is
// a matching that contains the largest possible
// number of edges. There may be many maximum
// matchings. The matching number of a graph is the
// size of a maximum matching. The problem is to find
// maximum matching of for given graph.
int match[150], V = 4, parent[150], base[150];
bool vis[150], temp[150], blossom[150];
vi vec[150];  queue<int> q;

int LCA(int a,int b){
  memset(temp,false,sizeof(temp));
  while(1){
    a = base[a];  temp[a] = true;
    if(match[a] == -1)break;
    a = parent[match[a]];
  }
  while(1){
    b = base[b];
    if(temp[b] == true)return b;
    b = parent[match[b]];
} }
void markPath(int v, int b, int children){
  for(; base[v]!=b; v=parent[match[v]]){
    blossom[base[v]] = blossom[base[match[v]]]=true;
    parent[v] = children;  children = match[v];
} }
int findPath(int root){
  memset(vis,false,sizeof(vis));
  memset(parent,-1,sizeof(parent));
  FOR(a,V)base[a] = a;  vis[root] = true;
  while(!q.empty())q.pop();  q.push(root);
  while(!q.empty()){
    int pos = q.front();  q.pop();
    int sz = vec[pos].size();
    FOR(a,sz){
      int nx = vec[pos][a];
      if(base[nx]==base[pos] || match[pos]==nx)
        continue;
      if(nx==root || match[nx]!=-1 &&                \
      parent[match[nx]]!=-1){
        int curbase = LCA(pos,nx);
        memset(blossom,false,sizeof(blossom));
        markPath(pos,curbase,nx);
        markPath(nx,curbase,pos);
        FOR(b,V) if(blossom[base[b]] == true){
          base[b] = curbase;
          if(vis[b]==false)vis[b] = true, q.push(b);
        } }
      else if(parent[nx]==-1){
        parent[nx] = pos;
        if(match[nx] == -1)return nx;
        nx = match[nx];  vis[nx] = true;
        q.push(nx);
  } } }
  return -1;
}
```

```
int edmonds(){
  int total = 0;  memset(match,-1,sizeof(match));
  FOR(a,V) if(match[a]==-1){
    int nx = findPath(a);
    while(nx!=-1){
      int pnx = parent[nx];  int ppnx = match[pnx];
      match[nx] = pnx;  match[pnx] = nx;  nx = ppnx;
  } }
  FOR(a,V) if(match[a] != -1)total++;
  return total/2;
}
```

AHO CORASICK

```
struct Trie{
  Trie *next[52], *fail;  vi label;  bool match;
  Trie(){
    fail = NULL;  match = false;  label.clear();
    memset(next,0,sizeof(next));
} };
void build_Trie(Trie** a, int val){
  int len = strlen(q_in);  Trie *p = *a;
  FOR(w,len){
    int idx = convert(q_in[w]);
    if(p->next[idx] == NULL)p->next[idx]=new Trie();
    p = p->next[idx];
  }
  p->label.pb(val);
}
void build_Automation(Trie* root){
  queue<Trie*> q;  Trie *state, *fail;
  root->fail = NULL;  q.push(root);
  while(!q.empty()){
    state = q.front();  q.pop();
    FOR(a,52){
      if(state->next[a] == NULL)continue;
      q.push(state->next[a]);
      fail = state->fail;
      while(fail!=NULL && fail->next[a]==NULL)
        fail = fail->fail;
      if(fail == NULL)state->next[a]->fail = root;
      else state->next[a]->fail = fail->next[a];
} } }
void search_pattern(Trie* root){
  Trie *p = root, *check;  int len = strlen(main_in);
  FOR(a,len){
    int nx = convert(main_in[a]);
    while(p!=root && p->next[nx]==NULL)p = p->fail;
    p = p->next[nx];
    if(p == NULL)p = root;
    check = p;
    while(check!=root && check->match==false){
      int sz = check->label.size();
      FOR(b,sz)mark[check->label[b]] = true;
      check->match = true;  check = check->fail;
} } }
void clear_Automation(Trie* root){
  queue<Trie*> q;  Trie *nx;  q.push(root);
  while(!q.empty()){
    nx = q.front();  q.pop();
    FOR(a,52) if(nx->next[a] != NULL)
      q.push(nx->next[a]);
    nx->label.clear();  delete nx;
} }
```

MAXIMUM CLIQUE

```
// max subgraph that form a complete graph in a Graph
class MaxClique{  public:
  static const int MV = 210;
  int V, el[MV][MV/30+1], dp[MV];
  int ans, s[MV][MV/30+1];  vi sol;

  void init(int v){
    V = v;  ans = 0;
    memset(el,0,sizeof(el)); memset(dp,0,sizeof(dp));
  }
  /* Zero Base */
  void addEdge(int u, int v){
    if(u > v)swap(u, v);
    if(u == v)return;
    el[u][v/32] |= (1<<(v%32));
  }
```

```
  bool dfs(int v, int k){
    int c = 0, d = 0;
    FOR(i,((V+31)/32)){
      s[k][i] = el[v][i];
      if(k != 1)s[k][i] &= s[k-1][i];
      c += __builtin_popcount(s[k][i]);
    }
    if(c == 0){
      if(k > ans)
        ans = k, sol.clear(), sol.pb(v), return 1;
      return 0;
    }
    FOR(i,((V+31)/32)) for(int a=s[k][i]; a; d++){
      if(k+(c-d) <= ans)return 0;
      int lb = a&(-a), lg = 0;  a ^= lb;
      while(lb != 1)lb = (unsigned int)(lb)>>1, lg++;
      int u = i*32+lg;
      if(k+dp[u] <= ans)return 0;
      if(dfs(u,k+1))sol.pb(v), return 1;
    }
    return 0;
  }
  int solve(){
    FORD(i,V)dfs(i,1), dp[i] = ans;  return ans;
} };
```

DSU ON TREE

```
// Given a tree, find LIS from all possible path
int sz[N];  vi lst[N];

void findsz(int idx, int par){ // precompute size
  sz[idx] = 1;
  FOR(i,lst[idx].size()){
    if(lst[idx][i] == par)continue;
    findsz(lst[idx][i],idx);
    sz[idx] += sz[lst[idx][i]];
} }
void addto(vi* vec, int a){ // add a to lis array vec
  vi::iterator p;
  p = lower_bound(vec->begin(),vec->end(),a);
  if(p == vec->end())vec->pb(a);
  else (*p) = a;
}
void combine(vi* a, vi *b){ // combine lis arr a & b
  for(int p=0; 1; p++){
    if(p >= b->size())break;
    else if(p >= a->size())a->pb((*b)[p]);
    else if(p<b->size())(*a)[p]=min((*a)[p],(*b)[p]);
} }
int ans = 0;  vi* alis[N], vi* alds[N]; // lis & lds
void dfs(int idx, int par, vi* lis, vi* lds){ // dfs
  int mx = -1, bigChild = -1;
  FOR(i,lst[idx].size()){ // find big child
    int u = lst[idx][i];
    if(u == par)continue;
    if(sz[u] > mx)mx = sz[u], bigChild = u;
  }
  addto(lis,idx);  addto(lds,-idx);
  if(bigChild == -1){ // is leaf
    ans = max(ans,(int)lis->size());
    ans = max(ans,(int)lds->size());
    alis[idx] = new vi();  alis[idx]->pb(idx);
    alds[idx] = new vi();  alds[idx]->pb(-idx);
  } else if(par!=-1 && lst[idx].size()==2){// 1 child
    dfs(bigChild,idx,lis,lds);
    alis[idx] = alis[bigChild];
    alds[idx] = alds[bigChild];
    addto(alis[idx], idx);
    addto(alds[idx], -idx);
  } else{ // more than one child, create copy array
    vi a = *lis;  vi b = *lds;
    dfs(bigChild,idx,&a,&b);
    alis[idx] = alis[bigChild];
    alds[idx] = alds[bigChild];
    addto(alis[idx],idx);  addto(alds[idx],-idx);
    combine(lis,alis[idx]);  combine(lds,alds[idx]);
    FOR(i,lst[idx].size()){ // dfs to other childs
      int u = lst[idx][i];
      if(u==par || u==bigChild)continue;
      a = *lis;  b = *lds;
      dfs(u,idx,&a,&b);
      addto(alis[u],idx);  addto(alds[u],-idx);
      combine(alis[idx],alis[u]);
      combine(alds[idx],alds[u]);
      if(i+1 < lst[idx].size())
        combine(lis,alis[u]), combine(lds,alds[u]);
      delete alis[u];  delete alds[u];
} } }
```

RECOVER MIN-VERTEX COVER from MAXIMUM MATCHING

Suppose you have left and right parts in bipartite graph and you found the maximum matching M. Let's define orientation of edges. Those edges that belong to M will go from right to left, all other edges will go from left to right. Now run DFS starting at all left vertices that are not incident to edges in M. Some vertices of the graph will become visited during this DFS and some not-visited. To get minimum vertex cover you take all visited right vertices of M, and all not-visited left vertices of M.

KMP ALGORITHM

```
char s[MAX+5], a[MAX+5];  int pos[MAX+5];

int KMP(int k){
  int la = strlen(a), ls = strlen(s), temp, res;
  temp = pos[0] = -1;
  FOR(i,la){
    while(pot>=0 && a[i]!=a[pot])temp = pos[temp];
    pos[i+1] = ++temp;
  }
  temp = res = 0;
  FOR(i,ls){
    while(temp>=0 && s[i]!=a[temp])temp = pos[temp];
    if(++temp == la)pot = pos[pot], return 1; //found
  }
  return 0; //not found
}
```

SUFFIX ARRAY

```
int dat[100005][25];  pair<pii,int> ret[100005];

void sparse(char* s, int n){
  FOR(i,100003) FOR(j,23)dat[i][j] = -1;
  FOR(j,20){
    //copy data
    ret[0] = mp(mp(-1,-1),-1);
    FOR(i,n){
      ret[i+1].se = i;
      if(j==0)ret[i+1].fi = mp((int)s[i],-1);
      else{
        int mask = (i+(1<<(j-1)))%n;
        ret[i+1].fi.fi = dat[i][j-1];
        ret[i+1].fi.se = dat[mask][j-1];
    } }
    sort(ret+1,ret+1+n);
    //iterate
    int pos=0;
    REP(i,1,n){
      if(ret[i-1].fi == ret[i].fi)
        dat[ret[i].se][j] = pos;
      else dat[ret[i].se][j] = ++pos;
} } }
int main(){
  int t,n;  char s[200005];
  scanf("%d",&t);
  while(t--){
    scanf("%d %s",&n,s);  sparse(s,n);  int res = 0;
    FOR(i,n){
      //compare with pos
      int posa = i, posr = res;
      FORD(j,20) if(n&(1<<j)){
        if(dat[posa][j] == dat[posr][j]){
          posa = (posa+(1<<j))%n;
          posr = (posr+(1<<j))%n;
          continue;
        }
        if(dat[posa][j] < dat[posr][j])res = i;
        break;
    } }
    printf("%d\n",res);
} }
```

ENHANCED SUFFIX ARRAY

```
#define MAX_N 100000
typedef struct{ int a[2], idx; } ttemp;
bool cmp(const ttemp& a, const ttemp& b){
  return (a.a[0]==b.a[0]) ? (a.a[1]<b.a[1]) :      \
    (a.a[0]<b.a[0]);
}
ttemp L[MAX_N], temp[MAX_N];
int step, p[20][MAX_N], lt, lq, a[MAX_N+1];

void rasort(ttemp x[], int n){
  int k = -1;  memset(a,0,sizeof(a));
  FOR(i,n)a[x[i].a[1]+1]++, k = max(k,x[i].a[1]+1);
  REP(i,1,k)a[i] += a[i-1];
  FORD(i,n)temp[ --a[x[i].a[1]+1] ] = x[i];
  memset(a,0,sizeof(a));  k = -1;
  FOR(i,n)a[temp[i].a[0]+1]++,k=max(k,temp[i].a[0]+1)
  REP(i,1,k)a[i] += a[i-1];
  FORD(i,n)x[ --a[temp[i].a[0]+1] ] = temp[i];
}
void generate(char s[], int p[][MAX_N]){
  int cnt;  FOR(i,lt)p[0][i] = s[i];
  for(step=1,cnt=1; (cnt>>1)<lt; step++,cnt<<=1){
    FOR(i,lt){
      L[i].a[0] = p[step-1][i];  L[i].idx = I;
      L[i].a[1] = (i+cnt<lt)?p[step-1][i+cnt]:-1;
    }
    rasort(L,lt);  //sort(L,L+lt); => n log^2 n
    FOR(i,lt)p[step][L[i].idx] = (i && L[i].a[0]    \
      == L[i-1].a[0] && L[i].a[1] == L[i-1].a[1])   \
      ? p[step][L[i-1].idx] : i;
  }
  step--;
}
char pattern[1100], text[100010];
//compare pattern suffix[idx]
inline int compare(int idx){
  int i = 0;
  for(;i<lq && idx<lt; i++,idx++){
    if(pattern[i] > text[idx])return -1;
    if(pattern[i] < text[idx])return 1;
  }
  return (i == lq) ? 0 : -1;
}
inline int lcp(int x, int y){
  if(x == y)return lt-x;
  int res = 0;
  for(int k = step-1; k>=0 && x<lt && y<lt; k--)
    if (p[k][x]==p[k][y])
      x+=(1<<k),y+=(1<<k),res+=(1<<k);
  return res;
}
//cari many pattern pada long text, preprocess text
int L = 0, R = lt-1, mid, found = false;
while(L <= R && !found){
  mid = (L+R)>>1;  int temp = compare(arr[mid]);
  if(temp==0)found = 1;
  else if(temp==1)R = mid-1;
  else L = mid+1;
}
//Longest palindrome
S = S + rev(S), edit LCP
for(int i=0;i<lt/2-1 ;i++){
  res >?= lcp(lt-i-1,i+1);
  if(i)res >?= 1 + lcp(lt-i,i+1);
}
//Distinct substring
if(lt&1)res = ((lt+1)>>1)*lt;
else res = (lt>>1) * (lt+1);
FOR(i,lt)arr[p[step-1][i]] = i;
FOR(i,lt-1)res -= lcp(arr[i],arr[i+1]);
//longest palindrome, maximum K change
int longpal(int left, int right, int k){
  int temp = 0, a;
  for(int j=0; j<=k; ){
    a = lcp(right,left);
    temp += 2*a;  left +=a;  right+=a;
    if(a==0){
      if(j==k)break;
      temp+=2;  left++;  right++;  j++;
    }
    if(left>= lt || right >= lt/2)break;
  }
  return temp;
}
```

EDIT DISTANCE

```
int dp[MAXN][MAXN];  char s1[MAXN], s2[MAXN];

int main(){
  int len1,len2;
  while(scanf("%d",&len1)!=EOF){
    if(len1>0)scanf("%s",s1);
    scanf("%d",&len2);
    if(len2>0)scanf("%s", s2);
    bool one = (len1==0)?1:0, two = (len2==0?1:0;
    FILL(dp,0);
    dp[0][0] = (s1[0]==s2[0]) ? 0 : 1;
    REP(i,1,len1-1)dp[i][0] = (s1[i]==s2[0]) ? i : \
      MIN(i+1,dp[i-1][0]+1);
    REP(i,1,len2-1) dp[0][i] = (s1[0]==s2[i]) ? i :\
      MIN(i+1,dp[0][i-1]+1);
    REP(i,1,len1-1) REP(j,1,len2-1){
      dp[i][j] = INF;
      if(s1[i] == s2[j])dp[i][j] <?= dp[i-1][j-1];
      dp[i][j] = MIN(dp[i][j],MIN(dp[i-1][j]+1,    \
        MIN(dp[i][j-1]+1,dp[i-1][j-1]+1)));
    }
    printf("%d\n",(one^two) ? (max(len1,len2)) :    \
      dp[len1-1][len2-1]);
} }
```

K-th PERMUTATION

```
string kthpermute(ll k, string s){
  int len = s.length();  char ss[len+2];  ll fac = 1;
  sprintf(ss,"%s",s.c_str());
  REP(i,2,len-1)fac = (ll)i * (ll)fac;
  FOR(i,len-1){
    int tidx=(k/fac)%(len-i);  char ch=ss[i+tidx];
    REV(j,i+tidx,i+1)ss[j] = ss[j-1];
    ss[i] = ch;  fac = fac/(ll)(len-i-1);
  }
  return (s = ss);
}
int main(){
  string s = "abcdefghijklmnopqrst";
  printf("%s\n",kthpermute(1234567654321,s).c_str());
}
```

Soal TIMO 2010

```
const ll mod = 1000000007;
ll  nck[100][100];  ll  memo[10][100];  int arr[10];

ll f(int x, int m){
  if(m ==  0)return 1;
  if(x == 10)return 0;
  if(memo[x][m] != -1)return memo[x][m];
  ll& ret = memo[x][m] = 0;
  FOR(i,min(m,arr[x]))ret+=f(x+1,m-i)*nck[m][i]%mod;
  return ret;
}
int main(){
  nck[0][0] = 1;
  REP(i,1,99){
    nck[i][0] = nck[i][i] = 1;
    REP(j,1,i-1)nck[i][j]=(nck[i-1][j]+nck[i-1]    \
    [j-1])%mod;
  }
  int T;
  scanf("%d",&T);
  while(T--){
    REP(i,1,9)scanf("%d",&arr[i]);
    int total = accumulate(arr+1,arr+10,0);
    ll ans = 0;
    REP(k,1,9) if(arr[k] != 0){
      arr[k]--;  memset(memo,-1,sizeof(memo));
      ll p = k;
      REP(len,1,total){
        ans = (ans+(f(1,len-1)*p)%mod) % mod;
        p = (p*10+k) % mod;
      }
      arr[k]++;
    }
```

```
    printf( "%I64d\n", ans );
} }
// Problem statement: Given the number of available
// digits of digit 1 to 9, sum all possible numbers
// that can be generated by those available digits.
```

MEET IN THE MIDDLE

```
long sumsLowerBound(long[] A, long B){
  int n = A.length;
  int sizeL = 1<<(n/2), sizeR = 1<<(n-n/2);
  long[] SumsL=new long[sizeL],SumsR=new long[sizeR];
  FOR(i,sizeL) FOR(j,n/2) if((i&(1<<j)) > 0)
    SumsL[i] += A[j];
  FOR(i,sizeR) FOR(j,n-n/2) if((i&(1<<j)) > 0)
    SumsR[i] += A[j+n/2];
  Arrays.sort(SumsL);  Arrays.sort(SumsR);
  int PL = 0, PR = sizeR-1;  long cur=Long.MIN_VALUE;
  while(PL<sizeL && PR >= 0){
    if(SumsL[PL]+SumsR[PR] <= B)
      cur = Math.max(cur,SumsL[PL++]+SumsR[PR]);
    else PR--;
  }
  return cur;
}
```

HUFFMAN CODES

```
#include <fstream>
int main(){
  priority_queue<ll,vector<ll>,greater<ll>> q;
  ifstream in("plank.in");
  ofstream out("plank.out");
  int N;  ll cost = 0;  in >> N;
  FOR(i,N){ ll x;  in >> x;  q.push(x); }
  for(;;){
    ll a = q.top();  q.pop();
    if(q.empty())break;
    ll b = q.top();  q.pop();
    cost += a+b;  q.push(a + b);
  }
  out << cost << "\n";
}
```

HUNGARIAN ALGORITHM

```
int cost[64][64], lvisit[64], rvisit[64];
int lvalue[64], rvalue[64], rmatch[64], n;

bool findmatch(int s){
  lvisit[s] = true;
  FOR(i,n){
    if(rvisit[i]||lvalue[s]+rvalue[i]!=cost[s][i])
      continue;
    rvisit[i] = true;
    if(rmatch[i]==-1 || findmatch(rmatch[i])){
      rmatch[i] = s;  return true;
  } }
  return false;
}
void normalize(){
  int down = INF;
  FOR(i,n) if(lvisit[i]) FOR(j,n) if(!rvisit[j])
    down = min(down,lvalue[i]+rvalue[j]-cost[i][j]);
  FOR(i,n){
    if(lvisit[i])lvalue[i] -= down;
    if(rvisit[i])rvalue[i] += down;
} }
int calc(){
  FOR(i,n){
    lvalue[i] = *max_element(cost[i],cost[i]+n);
    rvalue[i] = 0;
  }
  FOR(i,n)rmatch[i] = -1;
  FOR(i,n){
    FILL(lvisit,0);  FILL(rvisit,0);
    if(findmatch(i))continue;
    i--;  normalize();
  }
  int ret = 0;
  FOR(i,n)ret += cost[rmatch[i]][i];
  return ret;
}
```

STABLE MARRIAGE

```
int m,n, L[max_man][max_woman],R[max_woman][max_man];
int L2R[max_man],R2L[max_woman], p[max_man];

void stableMarriage(){
  static int p[128];
  memset(R2L,-1,sizeof(R2L));
  memset(p,0,sizeof(p));
  FOR(i,m) for(int man=i,hubby; man>=0; man=hubby){
    int wom;
    while(1){
      wom = L[man][p[man]++];
      if(R2L[wom]<0 || R[wom][man]>R[wom][R2L[wom]])
        break;
    }
    hubby = R2L[wom];  R2L[L2R[man]=wom] = man;
} }
```

CSOD

```
ll sum(ll a, ll b){
  a--;  return (b*(b+1)>>1) - (a*(a+1)>>1);
}
ll CSOD(LL n){
  ll i, m = n>>1, out2 = 0, j = sqrt(n);
  REP(i,2,j)out2=(n/i)*i;
  for(i--;j>=1;j--)out2 += (j*sum(i+1,n/j)), i=(n/j);
  return out2 -= ((n*(n+1)>>1)-1);
}
```

RDN (GOTO 01/01/0001)

```
// Rata Die day one is 0001-01-01
int rdn(int y, int m, int d){
  if(m < 3)y--, m += 12;
  return 365*y+y/4-y/100+y/400+(153*m-457)/5+d-306;
}
```

BIG INTEGER

```
const int maxlength=1000;
class bigint{
public:
  // 1. Basic
  int oper,length,a[maxlength];
  bigint(int=0);
  ~bigint();
  void check();
  void operator=(bigint m);
  void operator=(int m);
  void operator=(char *s);
  int max(int a,int b);
  // 2. Relational operations
  bool operator<(bigint m);
  bool operator<=(bigint m);
  bool operator>(bigint m);
  bool operator>=(bigint m);
  bool operator==(bigint m);
  bool operator!=(bigint m);
  // 3. Arithmetic operations
  bigint operator-();
  bigint operator+(bigint m);
  void operator+=(bigint m);
  bigint operator-(bigint m);
  void operator-=(bigint m);
  bigint operator*(bigint m);
  bigint operator*(int m);
  void operator*=(bigint m);
  void operator*=(int m);
  bigint operator/(bigint m);
  bigint operator/(int m);
  void operator/=(bigint m);
  void operator/=(int m);
  bigint operator%(bigint m);
  bigint operator%(int m);
  void operator%=(bigint m);
  void operator%=(int m);
};
  // 4. Input/ Output
bool read(bigint &m);
void write(bigint m);
void swrite(char *s,bigint m);
void writeln(bigint m);
  // 5. Mathematical functions
bigint abs(bigint m);
bigint sqr(bigint m);
bigint sqrt(bigint m);
```

```
bigint gcd(bigint a,bigint b);
bigint lcm(bigint a,bigint b);
bigint max(bigint a,bigint b);
bigint min(bigint a,bigint b);
bigint fpow(bigint a,bigint b);

// B. Class Implementation
// 1. Basic
bigint::bigint(int v){
  (*this) = v;
  this->check();
}
bigint::~bigint(){}
  void bigint::check(){
    for(; length>0 && a[length]==0; length--);
    if(length==0)oper=1;
}
void bigint::operator=(bigint m){
  oper = m.oper;  length = m.length;
  memcpy(a,m.a,maxlength*sizeof(int));
  this->check();
}
void bigint::operator=(int m){
  oper = (m>0)?1:-1;  m = abs(m);
  memset(a,0,maxlength*sizeof(int));
  for(length=0; m>0; m/=10000)a[++length] = m%10000;
  this->check();
}
void bigint::operator=(char *s){
  int i,L;  (*this)=0;
  if(s[0]=='-' || s[0]=='+'){
    if(s[0] == '-')oper=-1;
    L = strlen(s);  FOR(i,L)s[i] = s[i+1];
  }
  L = strlen(s);  length = (L+3)/4;
  FOR(i,L)a[(L-i+3)/4] = a[(L-i+3)/4]*10+(s[i]-48);
  this->check();
}
int bigint::max(int a,int b){return (a>b)?a:b;}

// 2. Relational operations
bool bigint::operator<(bigint m){
  if(oper!=m.oper)return oper<m.oper;
  if(length!=m.length)
    return oper*length<m.length*oper;
  REV(i,length,1) if(a[i] != m.a[i])
    return a[i]*oper < m.a[i]*oper;
  return false;
}
bool bigint::operator<=(bigint m){return !(m<(*this));
bool bigint::operator>(bigint m){ return m<(*this); }
bool bigint::operator>=(bigint m){return !((*this)<m);
bool bigint::operator==(bigint m){
  return (!((*this)<m)) && (!(m<(*this))); }
bool bigint::operator!=(bigint m){
  return ((*this)<m) || (m<(*this)); }

// 3. Arithmetic Operations
bigint bigint::operator-(){
  bigint c = (*this);  c.oper = -c.oper;
  c.check();  return c;
}
bigint bigint::operator+(bigint m){
  if(m.length == 0)return (*this);
  if(length == 0)return m;
  if(oper == m.oper){
    bigint c;  c.oper=oper;
    c.length = max(length,m.length)+1;
    for(int i=1,temp=0; i<=c.length; i++)
      c.a[i] = (temp=(temp/10000+a[i]+m.a[i]))%10000;
    c.check();  return c;
  }
  return (*this)-(-m);
}
bigint bigint::operator-(bigint m){
  if(m.length == 0)return (*this);
  if(length == 0)return (-m);
  if(oper == m.oper){
    bigint c;
    if(abs(*this)>=abs(m)){
      c.oper = oper;  c.length=length;
      for(int i=1,temp=0; i<=length; i++)
        c.a[i]=((temp=(-int(temp<0)+a[i]-m.a[i]))  \
        +10000)%10000;
      c.check();
      return c;
    }
    return -(m-(*this));
  }
  return (*this)+(-m);
}
bigint bigint::operator*(bigint m){
  bigint c;  c.oper = oper*m.oper;
  c.length = length+m.length;
  REP(i,1,m.length){
    int number = m.a[i], j, temp = 0;
    REP(j,1,length)c.a[i+j-1] += number*a[j];
    if(i%10==0 || i==m.length) REP(j,1,c.length)
      c.a[j] = (temp=(temp/10000)+c.a[j]) % 10000;
  }
  c.check();  return c;
}
bigint bigint::operator*(int m){
  if(m<0)return -((*this)*(-m));
  if(m>100000)return (*this)*bigint(m);
  bigint c;  c.length=length+2; c.oper=oper; int t=0;
  REP(i,1,c.length)c.a[i]=(t=(t/10000+a[i]*m))%10000;
  c.check();  return c;
}
bigint bigint::operator/(bigint m){
  if(m.length==0){
    printf("Division by zero.\n"); exit(0);
  }
  if(abs(*this)<abs(m))return 0;
  bigint c, left; c.oper = oper/m.oper;  m.oper = 1;
  c.length = length-m.length+1;
  left.length = m.length-1;
  memcpy(left.a+1,a+length-left.length+1,          \
    left.length*sizeof(int));
  REV(i,c.length,1){
    left = left*10000+a[i];
    int head = 0, tail = 10000, mid;
    while(head+1<tail){
      mid = (head+tail)/2;
      if(m*mid <= left)head = mid;
      else tail = mid;
    }
    c.a[i] = head;  left -= m*head;
  }
  c.check();  return c;
}
bigint bigint::operator/(int m){
  if(m<0)return -((*this)/(-m));
  if(m>100000)return (*this)/bigint(m);
  bigint c;  c.oper=oper;  c.length=length;  int t=0;
  REV(i,c.length,1)c.a[i] = (t=(t%m*10000+a[i]))/m;
  c.check();  return c;
}
bigint bigint::operator %(bigint m){
  return (*this)-((*this)/m)*m; }
bigint bigint::operator%(int m){
  if(m<0)return -((*this)%(-m));
  if(m>100000)return (*this)%bigint(m);
  int t=0;
  REV(i,length,1)t = (t*10000+a[i]) % m;
  return t;
}
void bigint::operator+=(bigint m){ (*this)=(*this)+m;}
void bigint::operator-=(bigint m){ (*this)=(*this)-m;}
void bigint::operator*=(bigint m){ (*this)=(*this)*m;}
void bigint::operator/=(bigint m){ (*this)=(*this)/m;}
void bigint::operator%=(bigint m){ (*this)=(*this)%m;}
void bigint::operator*=(int m){ (*this) = (*this)*m; }
void bigint::operator/=(int m){ (*this) = (*this)/m; }
void bigint::operator%=(int m){ (*this) = (*this)%m; }

// 4. Input/Output
bool read(bigint &m){
  char s[maxlength*4+10];
  if(scanf("%s",&s)==-1)return false;
```

```
  for(int i=0;s[i];i++) if(!(s[i]>='0' && s[i]<='9' \
    || (s[i]=='+'||s[i]=='-') && i==0))return false;
  m=s;
  return true;
}
void swrite(char* s, bigint m){
  int L = 0;
  if(m.oper == -1)s[L++] = '-';
  sprintf(s+L,"%d",m.a[m.length]);
  for(; s[L]!=0; L++);
  REV(i,m.length-1,1)sprintf(s+L,"%04d",m.a[i]),L+=4;
  s[L] = 0;
}
void write(bigint m){
  if(m.oper == -1)printf("-");
  printf("%d",m.a[m.length]);
  REV(i,m.length-1,1)printf("%04d",m.a[i]);
}
void writeln(bigint m){ write(m);  printf("\n"); }

// 5. Mathematical functions
bigint sqr(bigint m){ return m*m; }
bigint sqrt(bigint m){
  if(m.oper<0 || m.length==0)return 0;
  bigint c, last, now, templast;
  c.length = (m.length+1)/2;
  c.a[c.length] = int(sqrt((double)m.a[c.length*2]  \
    *10000+m.a[c.length*2-1])+1e-6);
  templast.length = c.length*2;
  templast.a[c.length*2-1] = (c.a[c.length]*        \
    c.a[c.length])%10000;
  templast.a[c.length*2] = (c.a[c.length]*          \
    c.a[c.length])/10000;
  templast.check();
  REV(i,c.length-1,1){
    last = templast;
    int head = 0, tail = 10000, mid, j, temp;
    while(head+1<tail){
      mid = (head+tail)/2;  now = last;
      now.a[2*i-1] += mid*mid;
      REP(j,i+1,c.length)now.a[i+j-1]+=mid*c.a[j]*2;
      now.length++;
      for (j=2*i-1,temp=0;j<=now.length;j++)
        now.a[j]=(temp=(temp/10000+now.a[j]))%10000;
      now.check();
      if(now<=m)templast = now, head=mid;
      else tail = mid;
    }
    c.a[i] = head;
  }
  c.check();  return c;
}
bigint abs(bigint m){
  bigint c = m;  c.oper=abs(c.oper);
  c.check();  return c;
}
bigint gcd(bigint a,bigint b){
  return b==0?a:gcd(b,a%b); }
bigint lcm(bigint a,bigint b){ return a*(b/gcd(a,b));}
bigint max(bigint a,bigint b){ return a>b?a:b; }
bigint min(bigint a,bigint b){ return a<b?a:b; }
bigint fpow(bigint a,bigint b){
  bigint zero = 0, one = 1, two = 2;
  if(b == zero)return one;
  if(b == one)return a;
  bigint half = fpow(a,b/two);
  if(b%two == 0)return half*half;
  else return half*half*a;
}
```

FAST FOURIER TRANSFORM + NTT

```
// For NTT, let p be modulo prime
// p = 2^k * c + 1
// need: size of polynom <= 2^k
// find r so that r^1 .. r^p-1 distinct <- call prr

// change for the following
// int theta = expmod(prr,(MOD-1)/n);
// if(sign == -1)theta = expmod(theta,MOD-2);
// w = expmod(theta,irev) // can be saved to array
// dont forget divmod and other mods

typedef complex<double> cpx;  typedef vector<cpx> vec;
void FFT(vec& a, int sign=1){
  int n = a.size(); // n should be a power of two
  double theta = sign*2*M_PI/n;
  for(int i=0,j=1; j<n-1; j++){
    for(int k=n>>1; k>(i^=k); k>>=1);
    if(j<i)swap(a[i],a[j]);
  }
  for(int m,mh=1; (m=mh<<1)<=n; mh=m){
    int irev = 0;
    for(int i=0; i<n; i+=m){
      cpx w = exp(cpx(0,theta*irev));
      for(int k=n>>2; k>(irev^=k); k>>=1);
      for(int j=i; j<mh+i; j++){
        int k = j+mh;
        cpx x = a[j]-a[k];
        a[j] += a[k];
        a[k] = w*x;
  } } }
  if(sign == -1) FOR(i,n)a[i] /= n;
}
void vector_multiply(vec& res, vec& a, vec& b){
  res.clear();
  int n = a.size()+b.size()-1;
  for(int i=1; i<n; i<<=1) if(n&i)n += i;
  a.resize(n);  b.resize(n);  FFT(a);  FFT(b);
  FOR(i,n)res.pb(a[i]*b[i]);  FFT(res,-1);
}
```

GAUSS ELIMINATION

```
typedef vector< vector<double> > Matrix;  Matrix M;

bool Gaussian () {
  int N = M.size();
  FOR(i,N){
    int idx = -1;
    REP(j,i,N-1) if(fabs(M[j][i])>EPS)idx = j,break;
    if(idx == -1){ return false; }
    REP(j,i,N)swap(M[i][j],M[idx][j]);
    REV(j,N,i)M[i][j] /= M[i][i];
    REP(j,i+1,N-1) REV(k,N,i)
      M[j][k] -= M[j][i]*M[i][k];
  }
  FORD(i,N+1) FOR(j,i) REV(k,N,i)
    M[j][k] -= M[j][i]*M[i][k];
  return true;
}
```

NUMBER THEORY

Bertrand's Ballot Theorem

Candidate A receives p votes and candidate B receives q votes,probability that A will be strictly ahead of B throughout count?

No Ties: $\frac{p-q}{p+q}$ Ties Allowed: $\frac{p+1-q}{p+1}$

Gambler's Ruin Theorem

Unfair:

$$P_1=\frac{1-\left(\frac{p}{q}\right)^{n_2}}{1-\left(\frac{p}{q}\right)^{n_1+n_2}} \qquad P_2=\frac{1-\left(\frac{q}{p}\right)^{n_1}}{1-\left(\frac{q}{p}\right)^{n_1+n_2}}$$

Fair:

$$P_1=\frac{n_2}{n_1+n_2} \qquad P_2=\frac{n_1}{n_1+n_2}$$

Pick Theorem

Let A be the area of a simply closed lattice polygon. Let B denote the number of lattice points on the Polygon edges and I the number of points in the interior of the polygon. Then **A=I+(1/2)B-1**

Forbenius Number

If GCD is 1 then required largest amount is (X*Y)-(X+Y) and total count is (X-1)*(Y-1)/2

Erdos Gallai Theorem

A sequence of non-negative integers $d_1 \geq \cdots \geq d_n$, can be represented as the degree sequence of a finite simple graph on n vertices if and only if $d_1 + \cdots + d_n$ is even

and $\sum_{i=1}^{k} d_i \le k(k-1) + \sum_{i=k+1}^{n} \min(d_i, k)$, holds every $1 \le k \le n$.

Derangement

```
ll der[15];   der[0] = 1;   der[1] = 0;
REP(i,2,10) der[i] = ((ll)i-1)*(der[i-1]+der[i-2]);
```

Bell Number

The number of ways a set of n elements can be partitioned into nonempty subsets

```
ll bell[25+5][25+5];   bell[0][0] = 1;
REP(i,1,maks){
  bell[i][0] = bell[i-1][i-1];
  REP(j,1,i) bell[i][j] = bell[i-1][j-1]+bell[i][j-1];
} // Example : n=10 ==> bell[10][0]
```

Stirling Number 2nd Kind

The number of ways a set of n elements into k subsets)
Recurrence : S(n,k) = k*S(n-1,k) + S(n-1,k-1)

```
int stir[n+1][k+1];
for (int i=0; i<=n; i++) stir[i][0] = 0;
for (int i=0; i<=k; i++) stir[0][k] = 0;
REP(i,1,n) REP(j,1,i){
  if (j == 1 || i == j) stir[i][j] = 1;
  else stir[i][j] = j*stir[i-1][j] + stir[i-1][j-1];
} // Example : n=3,k=1 : stir[3][1]
```

Carmichael Number

Composite numbers n such that a^(n-1) == 1 (mod n) for every a coprime to n.

```
// 561, 1105, 1729, 2465, 2821, 6601, 8911, 10585
// 15841, 29341, 41041, 46657, 52633, 62745, 63973
// 75361, 101101, 115921, 126217, 162401, 172081
// 188461, 252601, 278545, 294409, 314821, 334153
// 340561, 399001, 410041, 449065, 488881, 512461
```

Vandermonde Convolution

$$\binom{m+n}{r} = \sum_{k=0}^{r} \binom{m}{k}\binom{n}{r-k}, \qquad m,n,r \in \mathbb{N}_0,$$

Q-Vandermonde Convolution

$$\binom{n_1+\cdots+n_p}{m} = \sum_{k_1+\cdots+k_p=m} \binom{n_1}{k_1}\binom{n_2}{k_2}\cdots\binom{n_p}{k_p}.$$

int m,n

$$\sum_k \binom{r}{m+k}\binom{s}{n-k} = \binom{r+s}{m+n}$$

int l>=0, int m,n

$$\sum_k \binom{l}{m+k}\binom{s}{n+k} = \binom{l+s}{l-m+n}$$

int l>=0, int m,n

$$\sum_k \binom{l}{m+k}\binom{s+k}{n}(-1)^k = (-1)^{l+m}\binom{s-m}{n-l}$$

integers l,m,m>=0

$$\sum_{k\le 1} \binom{l-k}{m}\binom{s}{k-n}(-1)^k = (-1)^{l+m}\binom{s-m-1}{l-m-n}$$

integers l,m>=0,integers n>=q>=0

$$\sum_{o\le k\le l} \binom{l-k}{m}\binom{q+k}{n} = \binom{l+q+1}{m+n+1}$$

MATHEMATICS SEQUENCE FORMULA

$$\sum_{i=1}^{n} i^2 = \frac{n(n+1)(2n+1)}{6} \qquad \sum_{i=1}^{n} i^3 = \frac{n^2(n+1)^2}{4}$$

Triangular number sum

```
S(n) = ((n)(n+1)(n+2))/6
n = (sqrt(8x + 1) -1) / 2
```

$$\sum_{k=0}^{n} kCm = (n+1)C(m+1)$$

Faulhaber

$$\sum_{k=1}^{n} k^p = \frac{1}{p+1}\sum_{i=0}^{p}(-1)^i\binom{p+1}{i}B_i n^{p+1-i}$$

$$B_n = -\sum_{k=0}^{n-1}\binom{n}{k}\frac{B_k}{n-k+1} \quad for\ n>1; \quad 0\ for\ n=0$$

PRIME NUMBER LIST

```
10^ 1   7                  10^11   99999999977
10^ 2   97                 10^12   999999999989
10^ 3   997                10^13   9999999999971
10^ 4   9973               10^14   99999999999973
10^ 5   99991              10^15   999999999999989
10^ 6   999983             10^16   9999999999999937
10^ 7   9999991            10^17   99999999999999997
10^ 8   99999989           10^18   999999999999999989
10^ 9   999999937          10^19   9999999999999999961
10^10   9999999967
Below (2^31)-1: 2147483629 (2^31-1 is 8 Mersenne Prime)
Below (2^32)-1: 4294967291
Below (2^63)-1: 9223372036854775783
Below (2^64)-1: 18446744073709551557
```

CATALAN & SUPER CATALAN

```
ll superCatalan(int n){
  if(n<=2) return 1;
  return (3*(2*n-3)*sc(n-1)-(n-3)*sc(n-2)) / n;
} // 1,1,1,3,11,45, 197, 903, 4279, 20793, 103049
ll catalan(int n){
  if (n<=1) return 1;
  return c(n-1)*2*n*(2*n-1)/((n+1)*n);
} // 1,1,2,5,14,42, 132, 429, 1430, 4862, 16796, 58786
```

DIGIT COUNTING FACTORIAL

```
double memo[11000];
void start(){
  memo[1] = log10(1);
  REP(i,2,10000)memo[i]= memo[i-1]+(double)log10(i);
}
int solve(int n){ return (int)memo[n]+1; }
```

INT TO ROMAN

```
const string R[] = {"M","CM","D","CD","C","XC","L", \
  "XL","X","IX","V","IV","I"};
const int N[] = { 1000, 900, 500, 400, 100, 90, 50, \
  40, 10, 9, 5, 4, 1};

string toRoman(int x){
  if(x == 0)return "O"; // Not decimal 0 !
  int n = sizeof(N)/sizeof(int);
  string ret="";
  FOR(i,n) while(x>=N[i])x-=N[i],ret+=R[i];
  return ret;
}
```

nCk

```
ll res = 1;
if(n==0){ printf("%lld\n",res); continue; }
printf("%lld ",res);
for(int i=n,j=1; i>0,j<n; i--,j++){
  res = res*i/j;  printf("%lld ", res);
}
puts("1");

ll C(int n, int k){
  LL numerator = 1, denominator = 1, toMul, toDiv, i;
  if(k>n/2)k = n-k;
  REV(i,k,1){
    toMul = n-k+i;  toDiv = i;
    GCDdiv(toMul,toDiv);
    GCDdiv(numerator,toDiv);
    GCDdiv(toMul,denominator);
    numerator *= toMul;  denominator *= toDiv;
  }
  return numerator/denominator;
}
```

LINEAR DIOPHANTINE

```
// given a,b,c. This function will return true if
// found a solution. Need ext_euclid() .. ax + by = c;
// (d,x0,y0) = ext_euclid(a,b) .. x = x0 + (b/d)*n;
// y = y0 - (a/d)*n; n = 0,1,2, ...
bool linearDiop(int a, int b, int c, int& x, int& y){
  int d = ext_euclid(a,b,x,y);
  if(c%d != 0)return false;
  x *= c/d;  y *= c/d;  return true;
```

```
}
```

CRT dan kawan2

```
// Assume that f(0)=1 and 0^0=1. f(n) = (n%10)^f(n/10)
// for all n bigger than zero. Please calculate
// f(n)%m. (2 <= n, m <= 10^9, x^y means the y th
// power of x).
ll inverse(LL a, LL m){ return mPow(a,phi(m)-1,m); }
ll CRT( vector<pair<LL,int>> data ) {
  ll M = 1;
  For(auto x: data)M *= x.se;
  ll ret = 0LL;
  For(auto x: data){
    ll Mi = M/x.se;
    ll temp = x.fi*Mi*inverse(Mi,x.se);
    ret += temp;  ret %= M;
  }
  return ret;
}
int nums[16], n;  const int LIM=1<<10;
ll brute(ll base, int pos, ll M){
  if(pos==n)return base%M;
  int j = n;  ll pj = 1;
  while(j-1 >= pos){
    ll k = 1;
    FOR(loop,pj){
      K *= nums[j-1];
      if(k >= LIM)return 0;
    }
    pj = k;  j--;
  }
  return mPow(base,pj,M);
}
ll solve(int,ll);
ll doit(int,int);
pair<ll,int> solve2(int pos, ll M){
  ll a = nums[pos];
  if(GCD(a,M) == 1){
    ll b = solve(pos+1,phi((int)M));
    return mp(mPow(a,b,M),M);
  }
  return mp(brute(a,pos+1,M),M);
}
ll solve(int pos, ll M){
  if(M == 1)return 0;
  if(pos == n)return 1;
  if(nums[pos] == (ll)1)return a%M;
  if(nums[pos] == (ll)0)return 0;
  ll m = M;  vector< pair<ll,int> > data;
  for(ll i=2;i*i<=m;i++){
    if(m%i!=0)continue;
    ll p=1;
    while(m%i==0)m/=i,p*=i;
    data.pb(solve2(pos,p));
  }
  if(m>1)data.pb(solve2(pos, m));
  return CRT(data);
}
void normalize(){
  nums[n] = 1;  int newN = n;
  FOR(i,n){
    if(nums[i]==1)newN = i, break;
    else if(nums[i]==0){
      int ctr0 = 0;
      while(nums[i+ctr0] == 0)ctr0++;
      if(ctr0%2 == 0) nums[i] = 1, newN = i;
      else if(i-1 >= 0)nums[i-1] = 1, newN = i-1;
      break;
  } }
  n = newN;  if(nums[n-1] != 1)nums[n++] = 1;
}
ll doit(int x, int m){
  n=0;
  while(x)nums[n++] = x%10, x /= 10;
  normalize();
  return solve(0,m);
}
int main(){
  int x, m;
  FOR(ncase,getint()){
    scanf("%d %d",&x,&m);
    printf("%d\n",(int)doit(x,m));
} }

//CRT Willson
int inv(int a, int m){
  int m0 = m, t, q, x0 = 0, x1 = 1;
  if(m == 1)return 0;
  while(a > 1){
    q = a/m;  t = m;  m = a%m;  a = t;
    t = x0;  x0 = x1 - q*x0;  x1 = t;
  }
  if(x1 < 0)x1 += m0; // Make x1 positive
  return x1;
}
// k is size of num[] and rem[].  Returns the smallest
// number x such that:
//    x % num[0] = rem[0]
//    x % num[1] = rem[1]
//    ..................
//    x % num[k-2] = rem[k-1]
// Assumption: Numbers in num[] are pairwise coprime
// (gcd for every pair is 1)
int findMinX(int num[], int rem[], int k){
  int prod = 1, res = 0;
  FOR(i,k) prod *= num[i]; // Compute prod of nums
  FOR(i,k){
    int pp = prod/num[i];
    res += rem[i]*inv(pp, num[i])*pp;
  }
  return res % prod;
}
int driver(){
  int num[] = {3, 4, 5}, rem[] = {2, 3, 1};
  int k = sizeof(num)/sizeof(num[0]);
  cout << "x is "<< findMinX(num, rem, k);
}
```

EULER TOTIENT (PHI) FUNCTION

```
int phi(int n){
  if(n == 1)return 0;
  int ret = n;
  REP(i,2,n/i){
    if(n%i == 0)ret -= ret/i;
    while(n%i == 0)n /= i;
  }
  return ret - ( n!=1 ? ret/n : 0 );
}

// Sieve version
vi sieve_phi(int n){
  vector<bool> is_prime(n+1,true);  vi ret(n+1);
  FOR(i,n)ret[i] = i;
  REP(i,2,n) if(is_prime[i]) REPP(j,i,n,i){
    ret[j] -= ret[j]/i, is_prime[j] = false;
  }
  return ret;
}
void sievephi(int n){
  phi[1] = 0;  for(LL i=2; i<n; i++)phi[i] = i;
  for(ll i=2; i<=n; (i==(ll)2)?i++:i+=(ll)2)
    if(phi[i] == i) for(LL j=i; j<=n; j+=i)
        phi[j] = phi[j]*(ll)(i-(ll)1)/(ll)i;
  for(ll i=2;i<=n;i++) for(ll j=1,_n=n/i; j<=_n; j++)
    ans[i*j] += j*phi[i];
  for(ll i=2; i<=n; i++)ans[i] += ans[i-1];
}
int phi(int n){
  if(n == 1)return 0;
  int res = n;
  REPP(i,2,n/i,(i==2?1:2)){
    if(n%i)continue;
    while(n>1 && n%i==0)n/=i;
    res = (res%i) ? res*(i-1)/i : res/i*(i-1);
  }
  if(n > 1)res = res/n*(n-1);
  return res;
}
```

PRIME TEST

```
const int MAX = (int)1e8;  const int HALF = (int)5e7;
const int SQRT = 5000;  const int SIZE = MAX/16+2;
```

```
char SITO[SIZE];
#define NTH(n) ((n)&7)
#define ISPRIME(n) (SITO[(n)>>4]&(1<<NTH((n)>>1)))

bool isPrime(int n){
  if(n == 2)return true;
  else if(!(n&1))return false;
  else return ISPRIME(n);
}
void init(){
  FILL(SITO,0xFF);
  SITO[0] = 0xFE;
  REP(i,1,SQRT-1) if(ISPRIME(i<<1))
    REPP(j,2*i*(i+1),HALF-1,2*i+1)
      SITO[j>>3] &= ~(1 << NTH(j));
}
```

POLLARD RHO

```
ll pollard_rho(ll n){
  for(ll d=n,int c=2; d==n; c++){
    ll x = 2, y = 2;
    do{ // Russian peasant
      x = (mulmod(x,x,n)+c) % n;
      y = (mulmod(y,y,n)+c) % n;
      y = (mulmod(y,y,n)+c) % n;
      d = gcd(abs(x-y),n);
      if(d>1)break;
    } while(1);
  }
  pollard_rho(d);  pollard_rho(n/d);
}
```

MILLER RABIN

```
bool millerTest(ll p, int iteration){
  srand(time(NULL));
  if(p<2)return false;
  if(p!=2 && p%2==0)return false;
  ll s = p-1;
  while(s%2 == 0)s /= 2;
  FOR(i,iteration){
    ll a = rand()%(p-1)+1, temp = s;
    ll mod=fast_exp(a,temp,p);
    for(; temp!=p-1 && mod!=1 && mod!=p-1; temp*=2)
      mod = mulmod(mod,mod,p); // Russian peasant
    if(mod!=p-1 && temp%2==0)return false;
  }
  return true;
}
```

GENERALIZED LUCAS THEOREM

```
// Special Lucas : (n,k) % p^x
// fctp[n] =  Product of the integers less than or
// equal to n that are not divisible by p
// Precompute fctp
ll p;
ll E(ll n, int m){
  ll tot = 0;
  while(n!=0)tot += n/m, n /= m;
  return tot;
}
ll funct(ll n, ll base){
  ll ans = fast(fctp[base],n/base,base);
  return (ans*fctp[n%base]) % base;
}
ll F(ll n, ll base){
  ll ans = 1;
  while(n!=0)ans = (ans*funct(n,base))%base, n /= p;
  return ans;
}
ll special_lucas(ll n, ll r, ll base){
  p = fprime(base);
  ll pow = E(n,p)-E(n-r,p)-E(r,p);
  ll TOP = fast(p,pow,base)*F(n,base) % base;
  ll BOT = F(r,base)*F(n-r,base) % base;
  return (TOP*fast(BOT,totien(base)-1,base)) % base;
}
```

EXTENDED EUCLID

```
// given a and b for equation a*x + b*y = 1, extended
// Euclid will find the x and y and returns GCD(a,b)
int ext_euclid(int a, int b, int& x, int& y){
  if(b==0){ x=1;  y=0;  return a; }
  int ret = ext_euclid(b,a%b,x,y);
  int x1 = y, y1 = x-(a/b)*y;  x=x1;  y=y1;
  return ret;
}
```

SUM OF DIVISORS

```
// this function returns the sum of the divisors of n.
// i.e div(3) will return sum of the number of
// divisors up to 3.
ll div(int n){
  if(n==0)return 0;
  ll res = 0, i;
  for(i=2; i<=n/i; i++)res += n/i;
  i--;  res = (n*2)-(i*i)+(res*2);  return res;
}
```

RANGE SIEVE

```
void sieve(int L, int U, vector<int>& primes){
  memset(isnp,0,sizeof(isnp));
  int nto = U-L+1;
  for(auto nP: primes){
    if(nP > U/nP)break;
    int fnum = (L/nP)*nP;
    if(fnum < L)fnum += nP;
    if(fnum == nP)fnum += nP;
    REPP(k,fnum-L,nto,nP)isnp[k] = 1;
  }
  if(L <= 1)isnp[1-L] = 1;
  if(L <= 0)isnp[0-L] = 1;
}
```

MÖBIUS FUNCTION

```
int M[MAXN], SPF[MAXN];   // M=mobius, SPF=smallest pf
bool B[MAXN];  vi prima;  // B=is prime

void mobius(){
  prima.clear();  memset(B,true,sizeof(B));
  B[0] = B[1] = false;
  REP(i,2,MAXN-1){
    if(B[i])prima.pb(i), SPF[i] = i;
    FOR(auto j: prima){
      if(!(i*j<MAXN && j<=SPF[i]))break;
      B[i*j] = false;  SPF[i*j] = j;
  } }
  REP(i,1,MAXN-1){
    if(i == 1)M[i]=1;
    else if(SPF[i] == i)M[i] = -1;
    else if((i/SPF[i])%SPF[i] == 0)M[i] = 0;
    else M[i] = M[i/SPF[i]]*M[SPF[i]];
} }
```

GEOMETRIC TRANSFORMATION

Cermin: $\begin{bmatrix}\cos 2\theta & \sin 2\theta \\ \sin 2\theta & -\cos 2\theta\end{bmatrix}$ Rotasi: $\begin{bmatrix}\cos\theta & -\sin\theta \\ \sin\theta & \cos\theta\end{bmatrix}$

Dilatasi: $\begin{bmatrix}k & 0 \\ 0 & k\end{bmatrix}$ Gusuran/Sharing: $\begin{bmatrix}1 & x \\ y & 1\end{bmatrix}$

Regangan: $\begin{bmatrix}x & 0 \\ 0 & y\end{bmatrix}$

TRIGONOMETRIC EQUATIONS

```
Sin(3A)    = -4 Sin³ A + 3 Sin A
Cos(3A)    = 4 Cos³ A - 3 Cos A
Sin(A/2)    = ((1 - Cos A)/2)^0.5
Cos(A/2)    = ((1 + Cos A)/2)^0.5
Tan A + Tan B + Tan C = Tan A Tan B Tan C ;  A+B+C= PI

2 Sin A Cos B    = Sin(A+B) + Sin(A-B)
2 Cos A Sin B    = Sin(A+B) - Sin(A-B)
2 Cos A Cos B    = Cos(A+B) + Cos(A-B)
-2 Sin A Sin B    = Cos(A+B) - Cos(A-B)
```

Rumus Sin Cos Segitiga

```
a/Sin A = b/Sin B = c/Sin C
a² = b² + c² - 2 bc Cos A
L = ( ab Sin C )/2 = ( bc Sin A )/2 = ( ac Sin B )/2
L = (c^2 * sin A * sin B) / (2 * sin C)
```

Heron's Formula

```
L = (s(s-a)(s-b)(s-c))^0.5
s = (a+b+c)/2        r = L/ s        R = abc / 4L
```

Luas n-gon

```
L = ( n r r Sin(360/n) )/2
```

Lingkaran Singung Segitiga sisi:

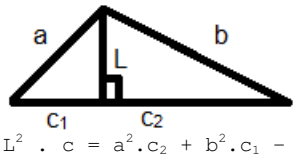
```
a: (s(s-b)(s-c)/(s-a))^0.5
b: (s(s-a)(s-c)/(s-b))^0.5
c: (s(s-a)(s-b)/(s-c))^0.5
```

$$m = \sqrt{\frac{2b^2 + 2c^2 - a^2}{4}}$$

Titik ke Garis

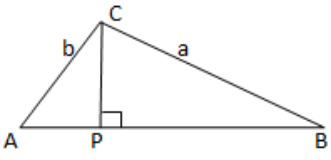
```
U = ((x3-x1)(x2-x1)+(y3-y1)(y2-y1))/((x2-x1)²+(y2-y1)²)
X = x1 + U(x2-x1)
Y = y1 + U(y2-y1)
```

Dalil Steward



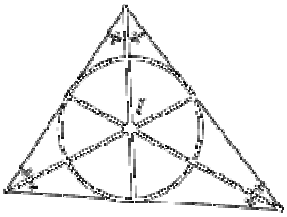
$L^2 . c = a^2.c_2 + b^2.c_1 - c_1.c_2.c$

Dalil Proyeksi

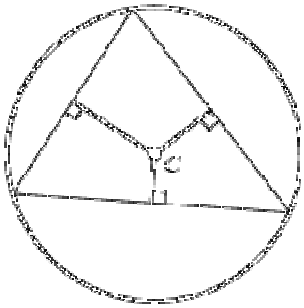


c

$AP = (b^2 + c^2 - a^2) / 2c$



incircle



circumcircle

Law of Tangents

$$\frac{a-b}{a+b} = \frac{\tan^1/_2(\alpha-\beta)}{\tan^1/_2(\alpha+\beta)}$$

$$\frac{b-c}{b+c} = \frac{\tan^1/_2(\beta-\gamma)}{\tan^1/_2(\beta+\gamma)}$$

Mollweide's Formula

$$\frac{a+b}{c} = \frac{\cos^1/_2(\alpha-\beta)}{\sin^1/_2\gamma}$$

$$\frac{a-c}{a+c} = \frac{\tan^1/_2(\alpha-\gamma)}{\tan^1/_2(\alpha+\gamma)}$$

POLYGON CENTROID AND AREA

$$C_x = \frac{1}{6A}\sum_{i=0}^{n-1}(x_i + x_{i+1})(x_i\ y_{i+1} - x_{i+1}\ y_i)$$

$$C_y = \frac{1}{6A}\sum_{i=0}^{n-1}(y_i + y_{i+1})(x_i\ y_{i+1} - x_{i+1}\ y_i)$$

$$A = \frac{1}{2}\sum_{i=0}^{n-1}(x_i\ y_{i+1} - x_{i+1}\ y_i)$$

(x(n),y(n)) == (x0,y0)

LINE SEGMENT INTERSECTION POINT

$$Y = \frac{\begin{vmatrix}\begin{vmatrix}x_1 & y_1\\ x_2 & y_2\end{vmatrix} & \begin{vmatrix}y_1 & 1\\ y_2 & 1\end{vmatrix}\\ \begin{vmatrix}x_3 & y_3\\ x_4 & y_4\end{vmatrix} & \begin{vmatrix}y_3 & 1\\ y_4 & 1\end{vmatrix}\end{vmatrix}}{\begin{vmatrix}\begin{vmatrix}x_1 & 1\\ x_2 & 1\end{vmatrix} & \begin{vmatrix}y_1 & 1\\ y_2 & 1\end{vmatrix}\\ \begin{vmatrix}x_3 & 1\\ x_4 & 1\end{vmatrix} & \begin{vmatrix}y_3 & 1\\ y_4 & 1\end{vmatrix}\end{vmatrix}} = \frac{\begin{vmatrix}\begin{vmatrix}x_1 & y_1\\ x_2 & y_2\end{vmatrix} & y_1 - y_2\\ \begin{vmatrix}x_3 & y_3\\ x_4 & y_4\end{vmatrix} & y_3 - y_4\end{vmatrix}}{\begin{vmatrix}x_1 - x_2 & y_1 - y_2\\ x_3 - x_4 & y_3 - y_4\end{vmatrix}},$$

$$X = \frac{\begin{vmatrix}\begin{vmatrix}x_1 & y_1\\ x_2 & y_2\end{vmatrix} & \begin{vmatrix}x_1 & 1\\ x_2 & 1\end{vmatrix}\\ \begin{vmatrix}x_3 & y_3\\ x_4 & y_4\end{vmatrix} & \begin{vmatrix}x_3 & 1\\ x_4 & 1\end{vmatrix}\end{vmatrix}}{\begin{vmatrix}\begin{vmatrix}x_1 & 1\\ x_2 & 1\end{vmatrix} & \begin{vmatrix}y_1 & 1\\ y_2 & 1\end{vmatrix}\\ \begin{vmatrix}x_3 & 1\\ x_4 & 1\end{vmatrix} & \begin{vmatrix}y_3 & 1\\ y_4 & 1\end{vmatrix}\end{vmatrix}} = \frac{\begin{vmatrix}\begin{vmatrix}x_1 & y_1\\ x_2 & y_2\end{vmatrix} & x_1 - x_2\\ \begin{vmatrix}x_3 & y_3\\ x_4 & y_4\end{vmatrix} & x_3 - x_4\end{vmatrix}}{\begin{vmatrix}x_1 - x_2 & y_1 - y_2\\ x_3 - x_4 & y_3 - y_4\end{vmatrix}}$$

LINE SEGMENT INTERSECTION

```
struct Vertex {int x, y;};
int GetArea2(Vertex& a, Vertex& b, Vertex& c) {
  return (b.x-a.x)*(c.y-a.y)-(c.x-a.x)*(b.y-a.y);
}
bool OnSegment(Vertex& a, Vertex& b, Vertex& c) {
  return(min(a.x, b.x) <= c.x && c.x <= max(a.x, b.x)
&& min(a.y, b.y) <= c.y && c.y <= max(a.y, b.y));
}
bool IsIntersect(Vertex& a, Vertex& b, Vertex& c,
Vertex& d) {
  int d1, d2, d3, d4;
  d1 = GetArea2(c, d, a);
  d2 = GetArea2(c, d, b);
  d3 = GetArea2(a, b, c);
  d4 = GetArea2(a, b, d);

  if(((d1 > 0 && d2 < 0) ||(d1 < 0 && d2 > 0)) &&((
d3 > 0 && d4 < 0) || d3 < 0 && d4 > 0)) {return true;}
  if(d1 == 0 && OnSegment(c, d, a)) {return true;}
  if(d2 == 0 && OnSegment(c, d, b)) {return true;}
  if(d3 == 0 && OnSegment(a, b, c)) {return true;}
  if(d4 == 0 && OnSegment(a, b, d)) {return true;}
  return false;
}
bool IsInside(Vertex& a, Vertex& lt, Vertex& rb) {
  return(min(lt.x, rb.x) <= a.x && a.x <= max(lt.x,
rb.x) && min(lt.y, rb.y) <= a.y && a.y <= max(lt.y,
rb.y));
}
// line intersection
struct Vertex {
  int x, y; Vertex result;
  Vertex operator - (Vertex& b) {
    result.x = x - b.x;
    result.y = y - b.y;
    return result;
  }
};
void GetIntersectionAttribute (Vertex& a, Vertex& b,
Vertex& c, Vertex& d, int& fN, int& sN, int& D) {
  fN = (d.x-c.x) * (a.y-c.y) - (d.y-c.y) * (a.x-c.x);
  sN = (b.x-a.x) * (a.y-c.y) - (b.y-a.y) * (a.x-c.x);
  D = (d.y-c.y) * (b.x-a.x) - (d.x-c.x) * (b.y-a.y);
}
double GetIntersectionTime (int N, int D) {
  return (double) N / (double) D;
}
double GetIntersectionX (Vertex& a, Vertex& b, double
t) {return a.x + (b - a).x * t;}
double GetIntersectionY (Vertex& a, Vertex& b, double
t) {return a.y + (b - a).y * t;}
```

SQUARE COLLISION

```
((x1>=x1t && x1<=x2t) && ((y1>=y1t && y1<=y2t) ||
(y2>=y1t && y2<=y2t) || (y1<=y1t && y2>=y2t))) ||
((x2>=x1t && x2<=x2t) && ((y1>=y1t && y1<=y2t) ||
(y2>=y1t && y2<=y2t) || (y1<=y1t && y2>=y2t))) ||
((x1<=x1t && x2>=x2t) && ((y1>=y1t && y1<=y2t) ||
(y2>=y1t && y2<=y2t) || (y1<=y1t && y2>=y2t)))
```

CLOSEST PAIR

```
const double inf = 10000.0;
const double eps = 1e-9;
typedef long long ll;
struct point {double x, y;};
int cmp(const point &a, const point &b) {
  return a.x < b.x;
```

```
}
inline double edist(point &a, point &b) {
  return sqrt((a.x-b.x)*(a.x-b.x) + (a.y-b.y)*(a.y-
b.y));
}
inline double dabs(double a) {
  return (a < 0) ? -a : a;
}
inline double dmin(double a, double b) {
  return (a < b) ? a : b;
}
int n, buff[10005], Y[10005], Yak[10005];
point X[10005];
inline double conquer(int l,int mid,int r,double dn) {
  for (int i=l,j=mid+1,k=l; i<=mid || j<=r; k++) {
    if ((j> r) || (i<=mid && X[Y[i]].y <X[Y[j]].y))
      buff[k] = Y[i], i++;
    else buff[k] = Y[j], j++;
  }
  int nak = 0;
  for (int i = l; i<=r; i++) {
    Y[i] = buff[i];
    if (dabs(X[mid].x - X[Y[i]].x) <= dn) {
      Yak[nak++] = Y[i];
    }
  }
  double ret = dn;
  for (int i = 0; i < nak; i++) {
    for (int j = i+1; j < nak; j++) {
      if (dabs(X[Yak[i]].y-X[Yak[j]].y)>ret)break;
      ret = dmin(ret,edist(X[Yak[i]], X[Yak[j]]));
    }
  }
  return ret;
}
inline double divide(int l, int r) {
  double ret = inf;
  if (l < r) {
    double dl = divide(l, (l+r)/2);
    double dr = divide((l+r)/2+1, r);
    ret = conquer(l, (l+r)/2, r, dmin(dl, dr));
  }
  return ret;
}
int main() {
  while (scanf("%d", &n) != EOF && n) {
    rep(i, n) scanf("%lf %lf", &X[i].x, &X[i].y);
    sort(X, X+n, cmp);
    rep(i, n) Y[i] = i;
    double ans = divide(0, n-1);
    if (dabs(ans-inf) <= eps) printf("INFINITY\n");
    else printf("%.4lf\n", ans);
  }
}
```

POLYGON CLASS

```
class Line {
  public :
  Point p1, p2;
  Line(){}
  Line(Point _p1, Point _p2):p1(_p1),p2(_p2){}
  bool intersect(double x1, double y1, double x2,
double y2, double x3, double y3, double x4, double
y4,double &x, double &y) {
    double nom = (x2-x1)*(y1-y3) + (y2-y1)*(x3-x1);
    double den = (x2-x1)*(y4-y3)-(x4-x3)*(y2-y1);
    if (den==0) return false;
    double b = nom/den;
    x = x3 + (x4-x3) * b;
    y = y3 + (y4-y3) * b;
    double a = (x2-x1 == 0) ? ((y-y1) / (y2-y1)) :
((x-x1) / (x2-x1));
    return 0<=a && a<=1 && 0<=b && b<=1;
  }
  bool intersect(Line &that, Point &p) {
    return intersect(p1.x, p1.y, p2.x, p2.y,
that.p1.x, that.p1.y, that.p2.x, that.p2.y, p.x, p.y);
  }
};
class Polygon {
  public :
  VP P;
  Polygon(){}
  Polygon(VP _P):P(_P){}
  void addPoint(Point p){P.push_back(p);}
  Polygon intersect(Polygon &that) {
    double qarea = that.area(), parea = area();
    VP &Q = that.P;
    set<Point> p;
    Point pt;
    REP(i,P.size()){
      int ni = (i+1)%P.size();
      Point p1(P[i].x, P[i].y);
      Point p2(P[ni].x,P[ni].y);
      Line L1(p1, p2);
      REP(j,Q.size()){
        int nj = (j+1)%Q.size();
        Point p3(Q[j].x, Q[j].y);
        Point p4(Q[nj].x,Q[nj].y);
        Line L2(p3, p4);
        if (L1.intersect(L2,pt)) p.insert(pt);
      }
    }
    REP(i,P.size()){
      double a = 0;
      REP(j,Q.size()) a += fabs(det(Q[j],
      Q[(j+1)%Q.size()], P[i])/2);
      if (fabs(a-qarea) < EPS) p.insert(P[i]);
    }
    REP(i,Q.size()){
      double a = 0;
      REP(j,P.size()) a += fabs(det(P[j],
      P[(j+1)%P.size()], Q[i])/2);
      if (fabs(a-parea) < EPS) p.insert(Q[i]);
    }
    return Polygon(cHull(VP(p.begin(),p.end())));
  }
  double area(){
    double ret = 0;
    REP(i, P.size()){
      int ni = (i+1)%P.size();
      ret += P[i].x * P[ni].y - P[i].y * P[ni].x;
    }
    return fabs(ret/2);
  }
};
int main(){
  for (int nA,nB,x,y; scanf("%d",&nA)!=EOF && nA; ){
    Polygon P, Q;
    REP(i,nA) {
      scanf("%d %d",&x,&y);
      P.addPoint(Point(x,y));
    }
    scanf("%d",&nB);
    REP(i,nB) {
      scanf("%d %d",&x,&y);
      Q.addPoint(Point(x,y));
    }
    Polygon I = P.intersect(Q);
    printf("%8.2lf\n",P.area()+Q.area()-2*I.area());
  }
  puts("");
  return 0;
}
```

PT TO LINE & LINE TO SEGMENT DIST

```
// Point to line
Returns the distance from p to the line defined by {a,
b}.The closest point on the line is returned through
(cpx,cpy). Does not work for degenerate lines (when
answer isundefined).
REQUIRES: dist()
double distToLine(
  double ax, ay, bx, by, px, py;
  double *cpx, double *cpy ) {
// Formula: cp = a + (p-a).(b-a) / |b-a| * (b-a)
double proj = ((px - ax)*(bx - ax) + (py - ay)*(by -
ay)) /((bx - ax) * (bx - ax) + (by - ay) * (by - ay));
    *cpx = ax + proj * ( bx - ax );
    *cpy = ay + proj * ( by - ay );
    return dist(px, py, *cpx, *cpy);
}
```

```
// Line to segment
Returns the distance from p to the line segment ab.
Theclosest point on ab is returned through (cpx, cpy).
Workscorrectly for degenerate line segments (a == b).
REQUIRES: dist(), distToLine()
double distToLineSegment(
  double ax, double ay,
  double bx, double by,
  double px, double py,
  double *cpx, double *cpy) {
  if ((bx - ax)*(px - ax)+(by - ay)*(py - ay)<EPS ) {
    *cpx = ax; *cpy = ay;
    return dist(ax, ay, px, py);
  }
  if ((ax - bx)*(px - bx)+(ay - by)*(py - by)<EPS ) {
    *cpx = bx; *cpy = by;
    return dist(bx, by, px, py);
  }
  return distToLine(ax, ay, bx, by, px, py, cpx,
cpy);
}
```

POLYGON DIAMETER

```
struct ConvexPolygon {
  Point *p;
  int n;
  ConvexPolygon(Point *pp, int nn) {p = pp; n = nn;}
double diameter() {
  int a, b;
  double d;
  double area1, area2;
  d = 0;
  if(n == 2) return getDist(p[0], p[1]);
  if(n == 3) return max(getDist(p[0],p[1]),
getDist(p[1], p[2]), getDist(p[0], p[2]));
  a = n - 1; b = 1;
  area1 = area(p[a], p[(a + 1) % n], p[(b + 1) % n]);
  area2 = area(p[a], p[(a + 1) % n], p[b]);
  while(area1 > area2) {
    b = (b + 1) % n;
    area1 = area(p[a], p[(a + 1) % n], p[(b+1)%n]);
    area2 = area(p[a], p[(a + 1) % n], p[b]);
  }
  while(b != 0) {
    a = (a + 1) % n;
    d = max(d, distance2Point(p[a], p[b]));
    area1 = area(p[a], p[(a + 1) % n], p[(b+1)%n]);
    area2 = area(p[a], p[(a + 1) % n], p[b]);
    while(area1 > area2) {
      b = (b + 1) % n;
      area1 = area(p[a], p[(a+1)%n], p[(b+1)%n]);
      area2 = area(p[a], p[(a + 1) % n], p[b]);
      if(b == 0) return d;
      d = max(d, distance2Point(p[a], p[b]));
    }
  }
  return d;
}};
```

UNION RECTANGLE INTERSECTION

```
struct event{ int ind, type;};
struct point{ double x,y; };

event events_v[350],events_h[350];
point rect[150][3];
bool in_set[350];
int n;

bool compare_x(event a,event b){
  return rect[a.ind][a.type].x<rect[b.ind][b.type].x;
}
bool compare_y(event a,event b){
  return rect[a.ind][a.type].y<rect[b.ind][b.type].y;
}

double union_area(int e){
  memset(in_set,false,sizeof(in_set));
  sort(events_v,events_v + e, compare_x);
  sort(events_h,events_h + e, compare_y);
  double area = 0;
  in_set[events_v[0].ind] = 1;
  for(int i=1;i<e;i++){
    event c = events_v[i];
    int cnt = 0;
    double delta_x = rect[c.ind][c.type].x -
rect[events_v[i-1].ind][events_v[i-1].type].x;
    double begin_y;
    if(delta_x == 0){
      in_set[c.ind] = (c.type == 0);
      continue;
    }
    for(int j=0;j<e;j++){
      if(in_set[events_h[j].ind] == 1){
        if(events_h[j].type == 0){
          if(cnt == 0) begin_y =
rect[events_h[j].ind][0].y;
          ++cnt;
        }
        elseif(--cnt == 0){
          double delta_y =
rect[events_h[j].ind][1].y - begin_y;
          area += delta_x * delta_y;
        }
      }
    }
    in_set[c.ind ] = (c.type == 0);
  }
  return area;
}
int main(){
  int tc = 0;
  while(cin>>n && n!=0){
    tc++;
    for(int a=0;a<n;a++){
      double x,y,r;
      cin>>x>>y>>r;
      rect[a][0].x = x-r;
      rect[a][0].y = y-r;
      rect[a][1].x = x+r;
      rect[a][1].y = y+r;
      events_v[2*a].ind = a;
      events_v[2*a].type = 0;
      events_v[2*a + 1].ind = a;
      events_v[2*a + 1].type = 1;
      events_h[2*a].ind = a;
      events_h[2*a].type = 0;
      events_h[2*a + 1].ind = a;
      events_h[2*a + 1].type = 1;
    }
    double area = union_area(2*n);
    printf("%d %.2lf\n",tc,area);
  }
}
```

GREAT CIRCLE DISTANCE

```
const double R = 6371.01;
double a=sin(plat)*sin(qlat);
double b=cos(plat)*cos(qlat)*cos(plong)*cos(qlong);
double c=cos(plat)*cos(qlat)*sin(plong)*sin(qlong);
return R * acos(a + b + c);
```

END OF TEAM NOTEBOOK

